

Criando Aulas Inovadoras com ArUcos e uma Câmera

Leonardo de Assis Junior Guimarães
Joab Felipe Lima João Pedro Queiroz
Állef Amorim Miguel Grigorio
Rafaela Crise Thaís Pedruzzi

Lab Penguin



INSTITUTO FEDERAL
Espírito Santo
Campus Guarapari

Escaneie o QR Code para
acessar a apostila online



PDF

Criando Aulas Inovadoras com ArUcos e uma Câmera

Leonardo de Assis Silva
Joab Felipe de Souza Lima
Állef Amorim Florindo
Rafaela da Silva Crise
Junior Guimarães de Oliveira
João Pedro Queiroz dos Santos
Miguel Grigório Araujo da Silva
Thaís Pedruzzi do Nascimento

Instituto Federal do Espírito Santo – IFES

Guarapari – ES

2026

Dados Internacionais de Catalogação na Publicação (CIP)
Instituto Federal do Espírito Santo – *Campus* Guarapari

C928 Criando aulas inovadoras com ArUcos e uma câmera / Leonardo de Assis Silva ... [et al.] – 2026.

100 p. : il.

ISBN: 978–65–5331–205–0

1. Tecnologia educacional. 2. Visão computacional. 3. Marcadores ArUco. I. Silva, Leonardo de Assis. II. Instituto Federal do Espírito Santo. III. Título.

CDD 371.33

Elaborado por Paulo Roberto Borghi Moreira CRB6-ES / 659

Esta obra está licenciada sob uma Licença Creative Commons
Atribuição-NãoComercial-CompartilhaIgual(CC BY-NC-SA).

© 2026 dos autores



1	Introdução	6
1.1	Motivação	6
1.2	Motivação descontraída	7
1.3	Introdução aos ArUcos	8
1.4	Conexão com a Base Nacional Comum Curricular (BNCC)	10
2	Preparando o computador	11
2.1	Instalando o Python e as bibliotecas	11
2.1.1	Instalação no Windows	12
2.1.2	Instalação no Linux	13
2.2	Instalando as bibliotecas	13
2.3	Onde escrever os códigos?	16
2.3.1	Instalação no Windows	16
2.3.2	Instalação no Linux	17
2.4	Vamos testar algo?	19
2.5	Executando um programa já criado	22
2.6	Cursos online gratuitos de Python	22
3	Exemplos com ArUcos	23
3.1	Chamada com ArUcos	23
3.2	Desenhando com ArUcos	26
3.3	Batalha de canhões com ArUcos	29

3.4	Museu de arte com ArUcos	36
3.5	Museu de arte com perspectiva e ArUcos	40
3.6	Jogo da memória com ArUcos	44
3.7	Abrindo um cofre com ArUcos	48
3.8	Quebra-cabeça com ArUcos	53
3.9	Realidade Aumentada com ArUcos	58
3.10	Mixer de áudios com ArUcos	66
3.11	Cenário em RA com ArUcos	70
3.12	Sistema solar em RA com ArUcos	78
3.13	Sistema cartesiano com ArUcos	86
4	FAQ - Perguntas que podem surgir	93
4.1	Como sei qual o tamanho do ArUco?	93
4.2	Como posso gerar um ArUco?	93
4.3	Minha câmera não está abrindo, o que faço?	94
4.4	Testei o exemplo e deu algum erro, o que aconteceu?	94
4.5	A IA sugeriu uma mudança para o meu código. Quando coleï, algumas linhas do código no editor de Python (VS Code) ficaram sublinhadas de vermelho, o que aconteceu?	94
4.6	Como baixar os arquivos do Github?	95
	Bibliografia	98

1. Introdução

1.1	Motivação	6
1.2	Motivação descontraída	7
1.3	Introdução aos ArUcos	8
1.4	Conexão com a Base Nacional Comum Curricular (BNCC)	10

Neste capítulo, você vai entender por que os marcadores ArUco podem ser uma ferramenta interessante para suas aulas e como eles funcionam na prática. A ideia é mostrar, de forma simples, como é possível transformar uma câmera comum em um recurso interativo para engajar os alunos.

1.1 Motivação

O uso de tecnologias digitais na educação tem se tornado cada vez mais importante para engajar os estudantes e aproximar o conteúdo escolar de sua realidade. No entanto, muitas dessas ferramentas ainda são vistas como complexas ou pouco acessíveis para o dia a dia da sala de aula, especialmente quando envolvem programação ou visão computacional. Diante disso, surge um desafio: como levar essas tecnologias para a prática pedagógica de forma simples e viável?

O Laboratório Penguin [5], localizado no IFES – Guarapari, atua nas áreas de robótica e visão computacional e, ao longo do ano, recebe visitas de alunos e professores da rede municipal. Essas visitas têm como objetivo apresentar o que é desenvolvido pelo grupo, incentivar o ingresso no Instituto Federal e despertar o interesse pela pesquisa e pelo desenvolvimento científico.

Durante essas atividades, a equipe do laboratório utiliza diferentes exemplos didáticos para aproximar seus projetos da realidade dos visitantes. Nesse contexto, os marcadores ArUco se destacam, pois estão presentes em diversas aplicações desenvolvidas pelo grupo e permitem demonstrar, de forma visual e interativa, conceitos que poderiam parecer abstratos.

Embora projetos como a construção de um robô hexápode ou a automação de um telescópio sejam bastante atrativos, eles geralmente exigem conhecimentos prévios de eletrônica e programação. Por outro lado, os marcadores ArUco acabam sendo uma alternativa simples, acessível e extremamente versátil. Com apenas uma câmera comum (como a de um celular, webcam ou notebook), um computador e marcadores visuais impressos em papel, é possível criar atividades interativas que estimulam a curiosidade, o raciocínio lógico e a participação ativa dos alunos.

Além disso, o uso de ArUcos possibilita a integração entre diferentes áreas do conhecimento. Professores de matemática podem explorar geometria e coordenadas espaciais; professores de física podem trabalhar conceitos de movimento e posição; enquanto professores de artes e linguagens podem desenvolver experiências interativas e criativas. Tudo isso com baixo custo e sem a necessidade de equipamentos sofisticados.

Com o objetivo de ampliar o alcance dessas experiências e torná-las acessíveis a mais professores, o Laboratório Penguin desenvolveu esta apostila. A proposta é demonstrar que é possível inovar na prática pedagógica utilizando ferramentas modernas de forma simples e acessível, por meio de exemplos práticos que podem ser adaptados a diferentes realidades escolares, com o apoio de ferramentas de inteligência artificial.

Este material também é aberto a contribuições. Sugestões e melhorias podem ser enviadas para leonardo.assis@ifes.edu.br, contribuindo para o aprimoramento contínuo deste documento.

Agora, vamos começar! No próximo tópico, você entenderá o que são os marcadores ArUco, como gerá-los e como utilizá-los em atividades práticas.

1.2 Motivação descontraída

Hoje em dia, a gente escuta o tempo todo que precisa usar mais tecnologia em sala de aula. E faz sentido! Ela realmente ajuda a engajar os alunos e aproximar o conteúdo da realidade deles. Mas, na prática, muita coisa ainda parece complicada demais, principalmente quando entra programação, visão computacional, essas coisas que parecem “de outro mundo”. E aí fica a dúvida: como levar algo assim para a sala de aula de forma simples?

Essa pergunta aparece bastante nas visitas que o Laboratório Penguin [5] recebe no IFES – Guarapari. Ao longo do ano, alunos e professores da rede municipal vão até o laboratório para conhecer os projetos de robótica e visão computacional que desenvolvemos por lá. A ideia é justamente mostrar que esse universo existe, que é acessível, e quem sabe despertar o interesse deles pela pesquisa e pela tecnologia.

Só que, durante essas visitas, a gente percebe uma coisa: muitos dos projetos são legais, chamam atenção... mas nem sempre são fáceis de reproduzir. Construir um robô hexápode ou automatizar um telescópio, por exemplo, exige um certo conhecimento de eletrônica e programação. Não é exatamente o tipo de coisa que dá pra levar direto pra sala de aula.

E é aí que os marcadores ArUco entram na história.

Eles começaram a aparecer em vários projetos do laboratório justamente porque são simples de usar e, ao mesmo tempo, muito poderosos. Com uma câmera comum (pode ser de celular, webcam ou notebook), um computador e alguns "QR-codes" impressos em papel, já dá pra fazer coisas bem interessantes. De repente, algo que parecia abstrato começa a ganhar forma na frente dos alunos, de maneira interativa.

E o mais interessante é que isso não fica preso a uma única disciplina. Dá pra usar em matemática, explorando geometria e espaço; em física, trabalhando movimento e posição; e até em atividades mais criativas, envolvendo arte ou linguagem. Tudo isso sem precisar de equipamentos caros ou uma estrutura complicada.

A partir dessas experiências, surgiu a ideia desta apostila. A intenção aqui não é trazer algo difícil ou cheio de teoria, mas mostrar caminhos simples para usar esse tipo de tecnologia em sala de aula. São exemplos práticos, pensados para que você possa adaptar à sua realidade, com o apoio de ferramentas atuais, como a inteligência artificial.

E vale dizer: este material não está “pronto e fechado”. Ele pode (e deve) melhorar com o

tempo. Se você tiver sugestões, ideias ou adaptações, pode enviar para leonardo.assis@ifes.edu.br. A ideia é construir isso de forma colaborativa.

Bom, chega de conversa — vamos começar. No próximo tópico, você vai entender o que são os marcadores ArUco, como gerar os seus e como começar a usá-los na prática.

1.3 Introdução aos ArUcos

Os ArUcos são marcadores visuais, parecidos com QR-codes, formados por quadrados pretos e brancos. Cada marcador tem um desenho único, que funciona como uma “identidade”. Ao utilizar uma câmera (de celular, notebook ou webcam), um programa de computador consegue reconhecer esse marcador na imagem. A Figura 1.1 mostra um exemplo de ArUco.

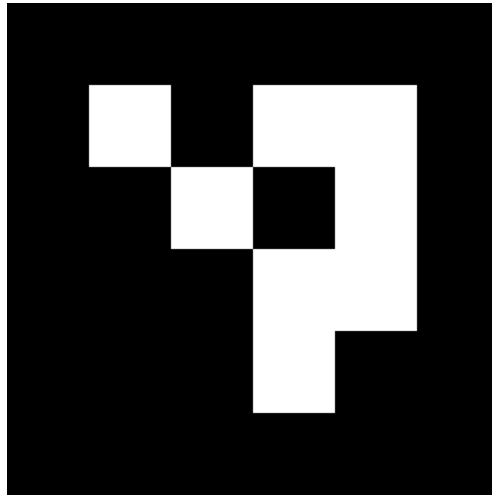


Figura 1.1: ArUco com indentificador (ID) 0.

Mas não é só isso. Além de identificar qual marcador está sendo visto, o sistema também consegue entender onde ele está e como ele está posicionado (inclinado, girado, mais perto ou mais longe da câmera). Na prática, isso permite fazer coisas bem interessantes, como acompanhar movimentos, medir posições ou até adicionar elementos virtuais na tela, como se estivessem no mundo real.



Figura 1.2: Realidade aumentada com ArUcos [3].

Como pode ser observado na Figura 1.2, assim como nós percebemos com os olhos, o computador também consegue identificar a inclinação e a rotação do ArUco. Isso permite, por exemplo, colocar um personagem digital sobre o marcador, fazendo com que ele acompanhe exatamente seus movimentos.

Uma das grandes vantagens dos ArUcos é a simplicidade. Eles podem ser gerados no computador, impressos em papel comum, feitos de EVA e usados com equipamentos que você provavelmente já tem disponível. Além disso, para que o processo de identificação funcione, não é necessário nenhum programa pago. É tudo gratuito!

Para a sala de aula, isso abre muitas possibilidades. Os alunos podem segurar e movimentar os marcadores, enquanto observam, em tempo real, como o computador reage. Isso torna a aula mais interativa e ajuda a transformar conceitos mais abstratos em algo visual e concreto.

Ao longo desta apostila, você verá exemplos práticos de como usar os ArUcos em atividades simples, utilizando Python, além de sugestões que podem ser adaptadas para diferentes níveis de ensino.

Python é o nome da linguagem de programação que utilizaremos nesta apostila. Assim como usamos o português ou o inglês para nos comunicar, usamos o Python para “conversar” com o computador e dizer o que ele deve fazer.



Em alguns exemplos, será necessário conhecer algumas informações sobre os ArUcos utilizados, como o ID de cada marcador e seu tamanho em centímetros.

Também é importante ter atenção ao tamanho dos marcadores em relação à distância da câmera. Se eles estiverem muito longe, marcadores muito pequenos podem não ser reconhecidos corretamente.

Uma forma simples de pensar nisso é a seguinte: quanto mais longe estamos de uma câmera, menores parecemos, e mais difícil fica distinguir detalhes. Com os ArUcos acontece a mesma coisa: quanto menor o marcador na imagem, mais difícil será para o computador identificá-lo.

Ok, ok! Você deve estar se perguntando: como posso gerar esses ArUcos?

Existem algumas formas simples de fazer isso. Podemos usar programas em Python ou até mesmo sites, como este [LINK](#). Nele, você escolhe o ArUco pelo seu ID e define o tamanho em milímetros antes de gerar a imagem, que pode ser baixada em PDF no tamanho de uma folha A4. O tamanho do ArUco é o tamanho do quadrado inteiro do marcador, não de partes internas. Ele define o tamanho real que ele terá quando for impresso (confira a Seção 4.1).

Além disso, assim como acontece com as palavras em um idioma, os ArUcos também estão organizados em “dicionários”. Cada dicionário define quais marcadores existem, seus respectivos IDs e o padrão de organização dos quadrados que formam cada marcador.

Por isso, ao trabalhar com ArUcos em Python, é necessário informar qual dicionário será utilizado. Na Figura 1.3, por exemplo, o dicionário 4×4 (50, 100, 250, 1000) gera marcadores com uma matriz interna de 4×4 quadrados, inserida em uma borda preta que forma uma grade maior.

Os números entre parênteses indicam a quantidade máxima de marcadores disponíveis naquele dicionário. Ou seja, dependendo da escolha, você poderá trabalhar com até 50, 100, 250 ou 1000 marcadores diferentes.

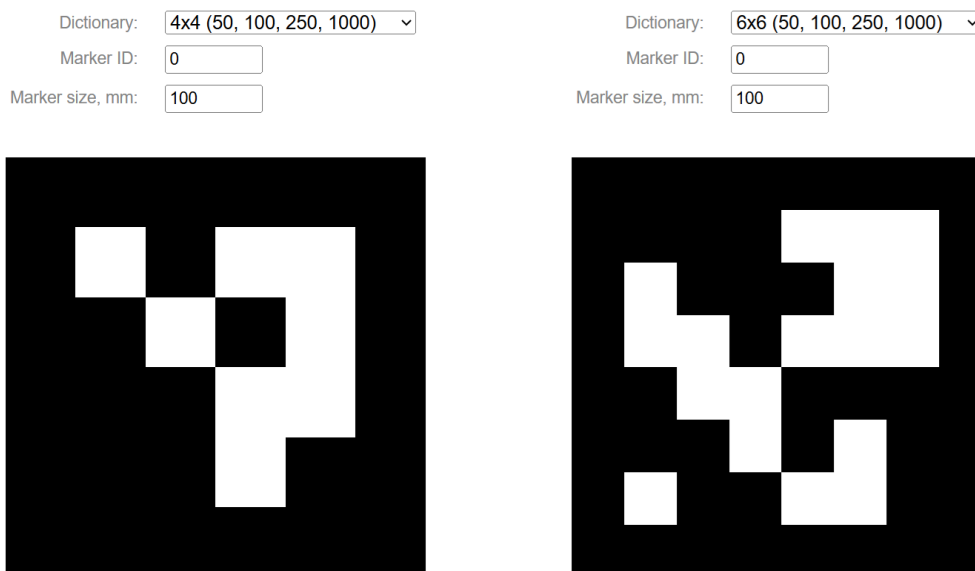


Figura 1.3: Simulação da escolha entre os dicionários 4×4 e 6×6 .

1.4 Conexão com a Base Nacional Comum Curricular (BNCC)

A simplicidade e a versatilidade dos marcadores ArUco permitem que os alunos transformem conceitos abstratos em experiências visuais e concretas. No entanto, nossa intenção é que, além do engajamento e da interatividade, o uso dessas ferramentas em sala de aula esteja diretamente relacionado às diretrizes pedagógicas da base curricular brasileira [1].

Por isso, em cada um dos exemplos práticos propostos, incluímos uma nota de “**Conexão com a BNCC**”, detalhando correlações iniciais com competências e habilidades oficiais. Note que não temos a pretensão de que esta seja uma lista exaustiva. O professor, com base em sua vivência em sala de aula e na realidade local, é incentivado a explorar outras conexões curriculares relevantes para o desenvolvimento dos alunos.

Além disso, todos os exemplos contemplam a Competência Geral 5 (Cultura Digital), ao promover o uso crítico e criativo de tecnologias digitais por meio de ferramentas de visão computacional. Por esse motivo, essa competência não será explicitamente listada nas notas de “**Conexão com a BNCC**” ao longo da apostila.



Conexão com a BNCC A nota estará escrita em um bloco como este.

2. Preparando o computador

2.1	Instalando o Python e as bibliotecas	11
2.1.1	Instalação no Windows	
2.1.2	Instalação no Linux	
2.2	Instalando as bibliotecas	13
2.3	Onde escrever os códigos?	16
2.3.1	Instalação no Windows	
2.3.2	Instalação no Linux	
2.4	Vamos testar algo?	19
2.5	Executando um programa já criado	22
2.6	Cursos online gratuitos de Python	22

Neste capítulo, você vai preparar o seu computador para começar a usar os marcadores ArUco. Vamos instalar o Python e as ferramentas necessárias, como o OpenCV, passo a passo. A ideia é deixar tudo pronto para que você possa acompanhar os exemplos e criar suas próprias atividades.

2.1 Instalando o Python e as bibliotecas

Antes de começarmos, é importante entender que precisaremos instalar algumas ferramentas no computador. Vamos instalar o Python (a linguagem de programação que utilizaremos) e também algumas bibliotecas, como o OpenCV e o Matplotlib, que permitem trabalhar com gráficos, imagens, câmeras e marcadores ArUco.

Bibliotecas são conjuntos de funções ou tarefas já programadas por outras pessoas que podemos usar para facilitar o desenvolvimento dos nossos programas. Uma boa forma de entender isso é pensar em uma calculadora. Você até pode fazer cálculos por sua conta, mas é muito mais prático usar a calculadora para obter o resultado diretamente, sem se preocupar com os detalhes do processo. Da mesma forma, em programação, usamos bibliotecas para evitar “reinventar a roda” e aproveitar soluções já prontas para tarefas comuns.



O processo de instalação é simples, mas pode variar de acordo com o sistema operacional. Nesta seção, você encontrará os passos para Windows e Linux. A ideia geral é a mesma, mas alguns comandos podem mudar, e isso será indicado quando necessário.

Não se preocupe se você não tem muita experiência com esse tipo de instalação. Vamos seguir passo a passo, de forma clara, para que tudo funcione corretamente ao final.

2.1.1 Instalação no Windows

Para começar a programar, precisamos instalar o Python no computador. Esse processo é simples e leva apenas alguns minutos.

Passo 1 (Baixar o Python) Acesse o site oficial do Python: <https://www.python.org>. No menu principal do site, há a opção Downloads. Ao passar o mouse sobre ela, aparecerá um botão indicando a versão mais recente do Python, por exemplo: Python 3.14.4, como mostrado na Figura 2.1.

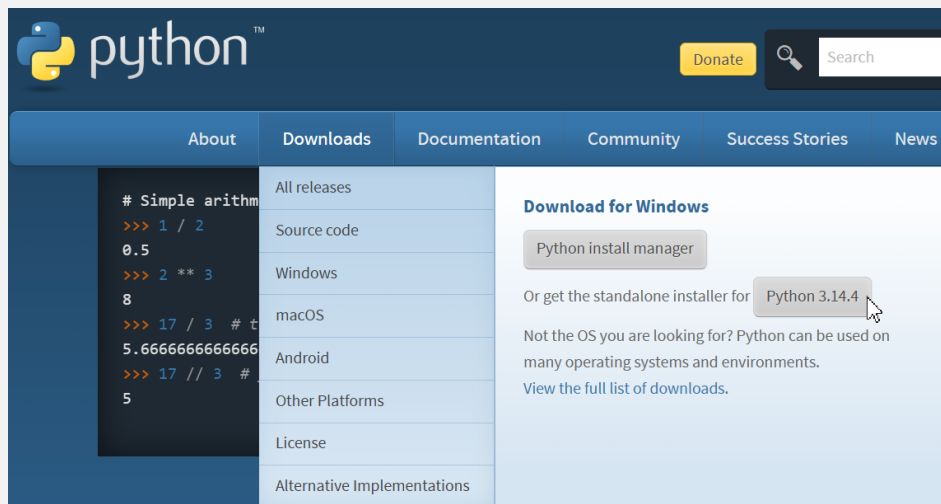


Figura 2.1: Local de download do Python.

Basta clicar neste local.

O site já sugere a versão mais recente compatível com o seu sistema.

Após terminar de baixar, vamos para o próximo passo.

Passo 2 (Executar o instalador) Após o download, abra o arquivo que foi baixado para iniciar a instalação.

Muita atenção neste momento: antes de clicar em “Install Now”, marque a opção Add Python to PATH.

Essa opção permite que você execute programas em Python de forma mais fácil no computador.

Depois disso, clique em *Install Now* e aguarde a instalação terminar.

Agora, vamos para o final.

Passo 3 (Verificar se deu tudo certo) Para conferir se o Python foi instalado corretamente:

Abra o menu iniciar do Windows;

Procure por Prompt de Comando (ou “cmd”) e abra;

Digite o seguinte comando e pressione Enter:

```
python3 --version
```

Se aparecer algo como **Python 3.x.x**, pronto! O Python foi instalado com sucesso.

Observação importante:

Caso o comando acima não funcione, é possível que a opção “Add Python to PATH” não tenha sido marcada durante a instalação. Nesse caso, será necessário reinstalar o Python marcando essa opção.

Excelente! O Python 3 já está instalado no seu computador.

Caso você não esteja utilizando Linux ou queira pular essa parte, pode seguir diretamente para a Seção 2.2, onde vamos aprender a instalar as bibliotecas necessárias no Python.

2.1.2 Instalação no Linux

Em muitos sistemas Linux, o Python já vem instalado por padrão. Por isso, antes de instalar qualquer coisa, vamos primeiro verificar se ele já está disponível no sistema.

Abra o terminal (*Ctrl + Alt + t*) e digite o seguinte comando:

```
python3 --version
```

Se aparecer algo como **Python 3.x.x**, significa que o Python já está instalado corretamente.

Caso o comando não funcione, ou caso você queira garantir que está utilizando a versão mais atual, podemos instalar ou atualizar o Python usando o gerenciador de pacotes/programas *apt* (em sistemas como Ubuntu e derivados).

Para isso, execute os seguintes comandos no terminal:

```
sudo apt update
sudo apt install python3 python3-pip
```

O primeiro comando atualiza a lista de pacotes do sistema, e o segundo instala o *Python3* junto com o gerenciador de pacotes *pip*, que será usado para instalar as bibliotecas que precisamos.

Ao final, você pode verificar novamente a instalação com:

```
python3 --version
```

Se tudo estiver correto, o Python estará pronto para uso no seu sistema.

2.2 Instalando as bibliotecas

Antes de instalar bibliotecas, é importante entender como identificar quando elas são necessárias. O programa em Python a seguir abre a imagem *imagem.jpg* armazenada no computador e exibe uma janela com essa imagem.

Para que isso seja possível, utilizamos funções fornecidas por duas bibliotecas externas. Isso pode ser observado no início do código, por meio das instruções `import cv2` e `import matplotlib`. Esses comandos indicam a utilização, respectivamente, das bibliotecas OpenCV (importada como `cv2`) e Matplotlib (importada como `plt`).

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("imagem.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.imshow(img)
plt.axis("off")
plt.show()
```

Portanto, para identificar quais bibliotecas precisam ser instaladas no computador, basta observar a lista de *imports* no início do programa. Cada linha `import` indica uma biblioteca externa necessária para que o código funcione corretamente.

Certo. Mas como podemos instalar essas bibliotecas e colocar o nosso programa para “rodar” de fato?

No caso do *Python3*, isso é feito por meio de um gerenciador de pacotes chamado *pip3*, que permite baixar e instalar bibliotecas diretamente da internet. Para instalar o OpenCV e o Matplotlib, basta abrir o terminal (ou prompt de comando) e executar os seguintes comandos:

```
pip3 install opencv-contrib-python
pip3 install matplotlib
```

Depois disso, o ambiente está pronto: basta executar o código/*script* novamente para que o programa consiga importar as bibliotecas e abrir a imagem corretamente.

Ok, você pode ter se perguntado: como saber o nome correto das bibliotecas para usar no *pip3*?

Uma dica prática é observar os *imports* do código e, quando encontrar bibliotecas que você ainda não instalou, consultar rapidamente uma ferramenta de busca ou uma IA com a pergunta: “como instalar as bibliotecas usadas nos *imports*?”. Veja o exemplo apresentado pelo print da Figura 2.2.

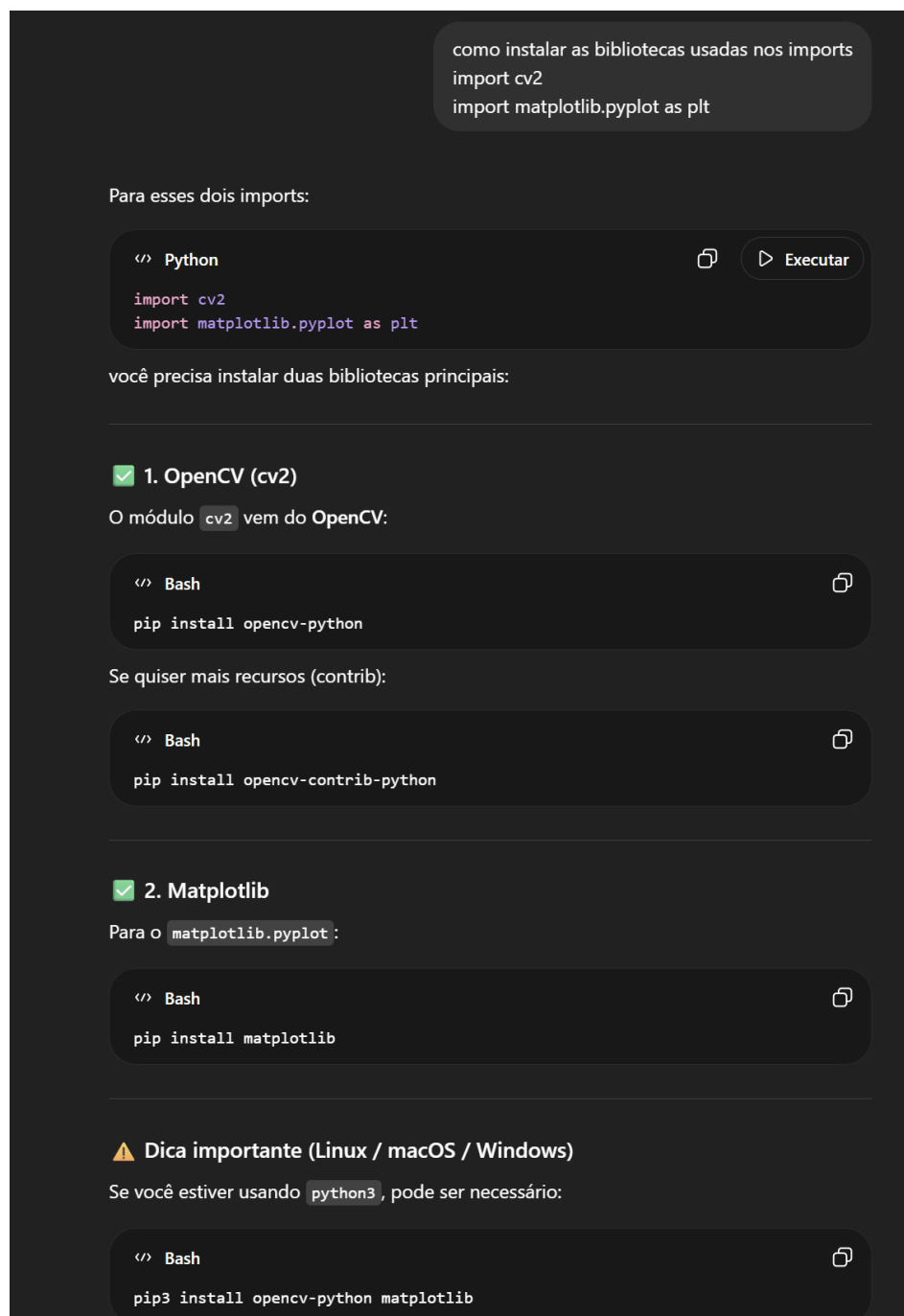


Figura 2.2: Conversa com uma IA sobre como instalar bibliotecas de *Python*.

Na conversa, a IA ainda comenta que, para que a instalação funcione, pode ser necessário utilizar *pip3* no lugar de *pip*, pois estamos usando *Python3* (exatamente como fizemos).

Comece instalando essas duas bibliotecas. Depois disso, sempre que encontrar alguma biblioteca que ainda não tenha instalado ao longo dos exemplos desta apostila, utilize uma IA como apoio para descobrir como fazer a instalação corretamente. Ela também pode ajudar caso ocorra algum erro: basta copiar a mensagem de erro e enviar para ela.

2.3 Onde escrever os códigos?

Instalei o Python e as bibliotecas, mas onde eu escrevo ou colo os códigos que vejo na apostila ou na internet?

Na prática, você pode até usar um bloco de notas simples (ou o gedit no Linux). O mais importante é salvar o arquivo com a extensão `.py`, e não `.txt`, para que o Python consiga executar o código.

Mas isso não é o mais indicado.

O ideal é usar um programa próprio para programação, como o VS Code. Ele ajuda bastante porque deixa o código colorido, mostra erros com mais clareza e facilita na hora de rodar os programas.

2.3.1 Instalação no Windows

Nesta seção, vamos instalar o VS Code (Visual Studio Code), que será o editor de código utilizado ao longo da apostila. Ele é um programa leve, gratuito e muito usado para escrever e executar códigos em Python.

Passo 1 (Baixar o VS Code) Acesse o site oficial do VS Code: <https://code.visualstudio.com/>.

Na página inicial, de cara, você verá um botão de Download para Windows. Clique nele e baixe o instalador.

Após terminar de baixar, vamos para o próximo passo.

Passo 2 (Executar o instalador) Após o download, abra o arquivo para iniciar a instalação.

Siga os passos normalmente clicando em “Next” até chegar na **seleção de tarefas adicionais**.

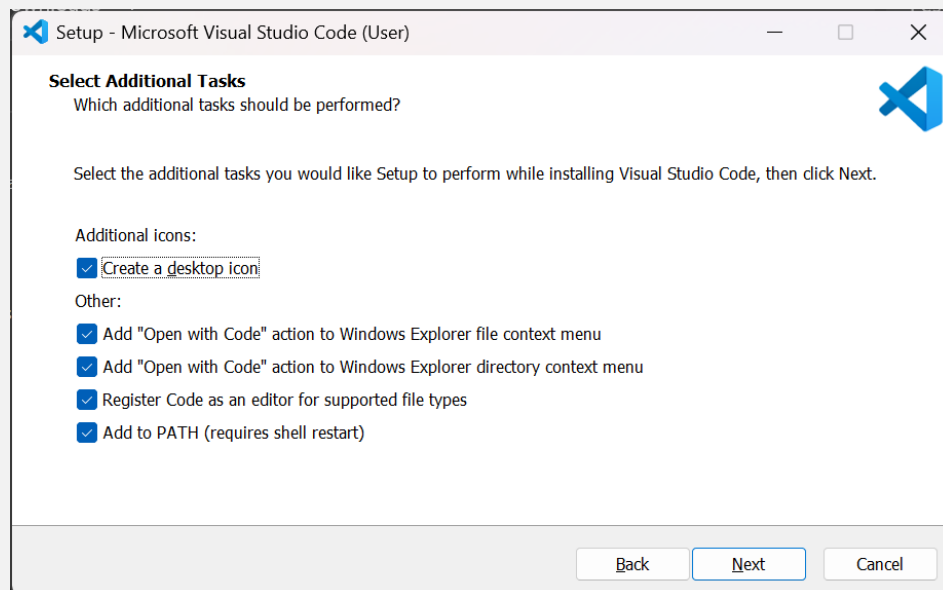


Figura 2.3: Configuração na hora da instalação do VS Code.

Lá, marque tudo conforme a Figura 2.3. Essas opções facilitam o uso do VS Code diretamente pelo terminal e pelos arquivos do sistema. Depois disso, clique em *Install* e aguarde o processo terminar.

Agora, vamos para o final.

Passo 3 (Instalação da extensão de Python no VS Code) Agora que o VS Code está instalado, precisamos configurá-lo para trabalhar com Python. Para isso, vamos instalar uma extensão chamada Python, que adiciona suporte à linguagem dentro do editor. Com o VS Code aberto, clique no menu da esquerda no ícone de extensões (uns quadradinhos empilhados), conforme a Figura 2.4. Na página que abrir, pesquise por Python e selecione a opção Python. Na área principal do VS Code, deve ter aparecido agora a página da extensão de Python. Clique em *Install*.



Figura 2.4: Configuração na hora da instalação do VS Code.

Após o término da instalação, o botão de *Install* deve mudar para *Uninstall*, indicando que tudo deu certo, conforme a Figura 2.4.

Pronto!

Agora o VS Code está configurado para trabalhar com Python. No próximo passo, vamos testar um primeiro código de exemplo com ArUcos e visão computacional, colocando em prática tudo o que instalamos até aqui.

2.3.2 Instalação no Linux

Nesta seção, vamos instalar o VS Code (Visual Studio Code) no Ubuntu. Ele será o editor de código utilizado ao longo da apostila para escrever e executar nossos programas em Python.

Passo 1 (Instalar o VS Code) A forma mais simples de instalar o VS Code no Ubuntu é utilizando o gerenciador de pacotes *snap*. No terminal, execute:

```
sudo snap install code --classic
```

Esse comando irá baixar e instalar automaticamente o VS Code.

Após terminar de instalar, vamos para o próximo passo.

Passo 2 (Abrir o VS Code) Após a instalação, você pode abrir o programa de duas formas:

1. Pelo terminal, digitando:

```
code
```

2. Ou procurando por “Visual Studio Code” no menu de aplicativos do sistema.

Agora, vamos para o final.

Passo 3 (Instalação da extensão de Python no VS Code) Agora que o VS Code está instalado, precisamos configurá-lo para trabalhar com Python. Para isso, vamos instalar uma extensão chamada Python, que adiciona suporte à linguagem dentro do editor. Com o VS Code aberto, clique no menu da esquerda no ícone de extensões (uns quadradinhos empilhados), conforme a Figura 2.5. Na página que abrir, pesquise por Python e selecione a opção Python. Na área principal do VS Code, deve ter aparecido agora a página da extensão de Python. Clique em *Install*.



Figura 2.5: Configuração na hora da instalação do VS Code.

Após o término da instalação, o botão de *Install* deve mudar para *Uninstall*, indicando que tudo deu certo, conforme a Figura 2.5.

Pronto!

Agora o VS Code está configurado para trabalhar com Python. No próximo passo, vamos testar um primeiro código de exemplo com ArUcos e visão computacional, colocando em prática tudo o que instalamos até aqui.

2.4 Vamos testar algo?

A partir daqui, vamos seguir um passo a passo que será utilizado em todos os exemplos desta apostila. Ou seja, sempre que você for criar um programa, você passará por essas etapas iniciais de preparação e execução.

A seguir, veremos um código que usa a câmera para reconhecer marcadores ArUco. Ao encontrar um marcador, o programa desenha um contorno ao redor dele e mostra seu número (ID) na tela.

Ao copiar um código diretamente do PDF, a formatação pode ser alterada. No Python, isso é um problema, pois os espaços no início das linhas (indentação) são essenciais para o funcionamento do programa.

Por isso, recomendamos que os códigos sejam acessados pelos links disponíveis no GitHub do laboratório. Lá, você poderá copiar o código corretamente, sem erros de formatação.

Sempre que aparecer este símbolo na apostila, ele indicará um link para o código original, como neste exemplo: [link do código](#).



```
import cv2

# Carrega o dicionário de ArUco (voce pode mudar se quiser outro tipo)
dicionario = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)

# Parametros do detector
parametros = cv2.aruco.DetectorParameters()

# Cria o detector
detector = cv2.aruco.ArucoDetector(dicionario, parametros)

# Abre a camera (0 = camera padrao do notebook)
cap = cv2.VideoCapture(0)

if not cap.isOpened():
    print("Erro ao abrir a camera")
    exit()

while True:
    ret, frame = cap.read()
    if not ret:
        print("Erro ao capturar imagem")
        break

    # Detecta os ArUcos
    corners, ids, _ = detector.detectMarkers(frame)
```

```
# Se encontrou algum marcador
if ids is not None:
    # Desenha os contornos e IDs
    cv2.aruco.drawDetectedMarkers(frame, corners, ids)

# Mostra a imagem
cv2.imshow("Deteccao de ArUco", frame)

# Pressione 'q' para sair
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Libera a camera e fecha tudo
cap.release()
cv2.destroyAllWindows()
```

Para executar esse código, primeiro crie uma pasta para o seu projeto. Neste exemplo, vamos chamá-la de Exemplo.

Em seguida, abra o VS Code e vá em File → Open Folder... (Arquivo → Abrir Pasta...) para abrir essa pasta.

Caso apareça uma mensagem perguntando se você confia nos autores da pasta, clique em “Yes, I trust the authors”.

Como a pasta ainda está vazia, será necessário criar um novo arquivo. Depois de criar esse arquivo, copie e cole nele o código apresentado anteriormente. A Figura 2.6 indica onde clicar para criar um novo arquivo dentro da pasta do projeto pelo VS Code.

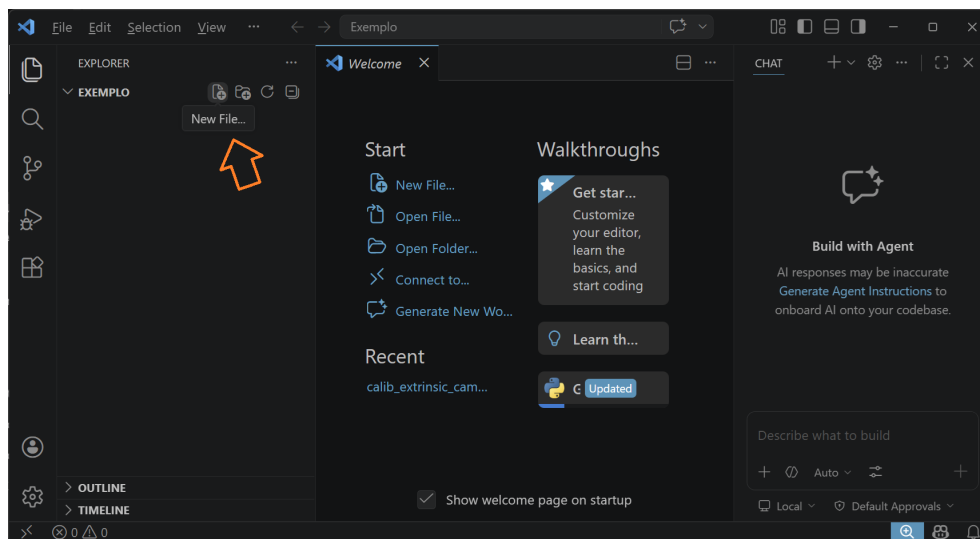


Figura 2.6: Como criar um arquivo na pasta do projeto.

Digite um nome para o arquivo e pressione Enter. Neste exemplo, vamos chamá-lo de `exemplo.py`.

Em seguida, copie e cole o código apresentado anteriormente dentro do arquivo e salve. Ao final, o VS Code deverá ficar semelhante ao mostrado na Figura 2.7.

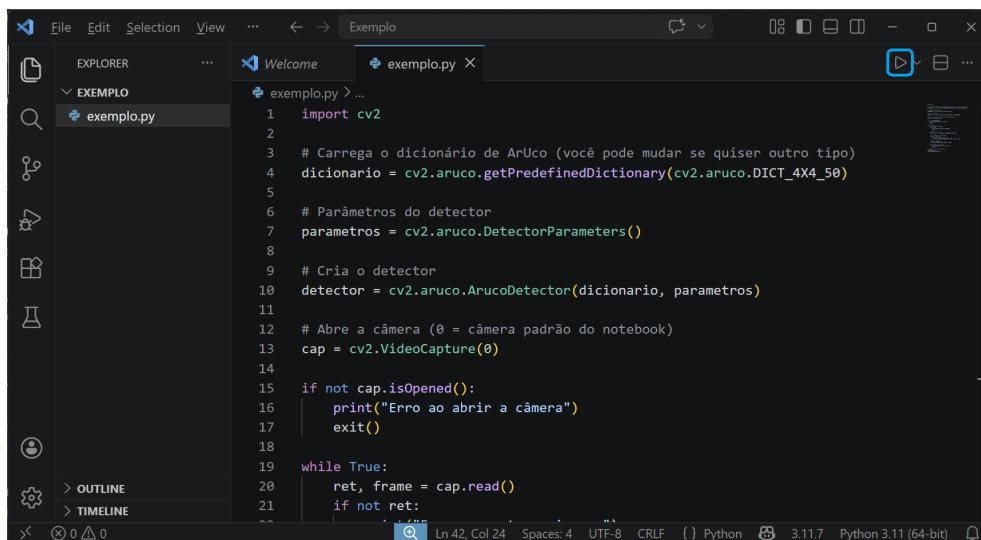


Figura 2.7: Arquivo de exemplo no VS Code.

Para executar o código, clique no botão *play* do VS Code (destacado em azul na Figura 2.7). Se tudo estiver correto, uma janela será aberta mostrando a imagem da sua câmera. Para encerrar a execução, pressione a tecla *q*, conforme indicado no código.

Em seguida, aponte um marcador ArUco para a câmera e observe o resultado, semelhante ao mostrado na Figura 2.8.

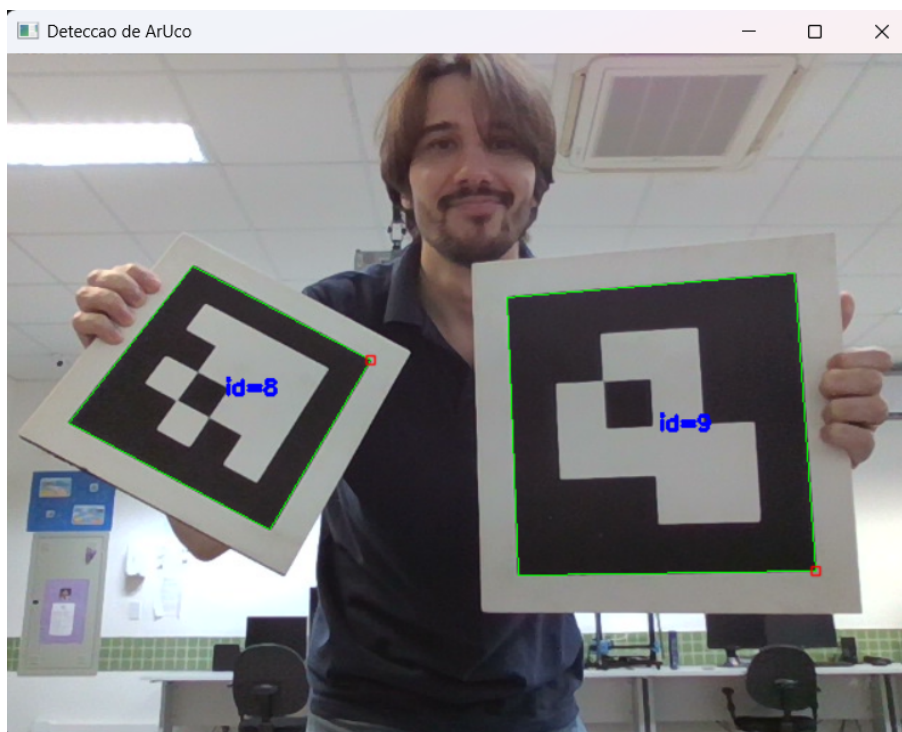


Figura 2.8: Resultado do programa de exemplo.

Como indicado no código, neste exemplo utilizamos o dicionário $4 \times 4_{50}$. Isso significa que o programa é capaz de reconhecer marcadores ArUco 4×4 com IDs de 0 até 49.

Faça o teste!

2.5 Executando um programa já criado

Se você já criou o código anteriormente e deseja executá-lo novamente, o processo é simples.

Abra o VS Code, vá em File → Open Folder... e selecione a pasta do seu projeto. Em seguida, clique no arquivo *exemplo.py* e pressione o botão *play* para rodar o programa novamente.

Em caso de dúvida, tente descrever claramente o que você quer entender e pergunte a uma IA. Você pode, por exemplo, colar o código em Python e pedir uma explicação passo a passo ou solicitar modificações para que ele realize outras tarefas, como faremos em alguns exemplos desta apostila.

2.6 Cursos online gratuitos de Python

Para os leitores que desejam aprofundar seus conhecimentos em programação Python, vamos deixar uma sugestão de um curso online gratuito no Youtube, no canal Curso em Vídeo [2]. Alguns dos alunos do nosso laboratório já utilizaram esse curso como apoio inicial para o aprendizado de Python e, por isso, acreditamos que ele pode ser usado como complemento aos conteúdos desta apostila.

Além disso, os leitores podem acessar outros materiais gratuitos de apoio desenvolvidos por alunos do curso de Engenharia Elétrica da UFES. O ambiente de tutoria disponível em Programming Tutor [7] oferece suporte complementar ao aprendizado de programação, enquanto o canal Python Tutoriais [6] disponibiliza videoaulas gratuitas sobre Python, cobrindo desde conceitos básicos até conteúdos mais avançados.

3. Exemplos com ArUcos

3.1	Chamada com ArUcos	23
3.2	Desenhando com ArUcos	26
3.3	Batalha de canhões com ArUcos	29
3.4	Museu de arte com ArUcos	36
3.5	Museu de arte com perspectiva e ArUcos	40
3.6	Jogo da memória com ArUcos	44
3.7	Abrindo um cofre com ArUcos	48
3.8	Quebra-cabeça com ArUcos	53
3.9	Realidade Aumentada com ArUcos	58
3.10	Mixer de áudios com ArUcos	66
3.11	Cenário em RA com ArUcos	70
3.12	Sistema solar em RA com ArUcos	78
3.13	Sistema cartesiano com ArUcos	86

Neste capítulo, você encontrará diversos exemplos práticos utilizando marcadores ArUco. A proposta é apresentar ideias simples e diretas que você pode testar no seu computador e, depois, adaptar para suas aulas. Ao explorar esses exemplos, você poderá entender melhor como os ArUcos funcionam na prática e se inspirar para criar suas próprias atividades com os alunos.

3.1 Chamada com ArUcos



Vamos fazer uma chamada automática utilizando ArUcos!

Cada aluno deve ter um marcador com um ID diferente. Execute o programa clicando em *play* e aponte a câmera para os alunos. Não é necessário que todos os ArUcos apareçam ao mesmo tempo: basta que cada marcador seja detectado pelo menos uma vez para que o aluno seja registrado como presente. Ao finalizar, pressione *q* para encerrar o programa e gerar automaticamente a lista de presença em um arquivo *.txt*.



O código da chamada pode ser encontrado neste [link](#).

```
import cv2
import cv2.aruco as aruco
import unicodedata

# =====
# Lista de chamada (ID : Nome)
# =====
lista_chamada = {
    17: "Allefe Florindo",
    35: "Rafaela Crise",
    49: "Joab Felipe",
    8: "Joao Pedro",
    9: "Miguel Grigorio"
}

def remover_acentos(texto):
    return ''.join(
        c for c in unicodedata.normalize('NFD', texto)
        if unicodedata.category(c) != 'Mn')

# Conjunto para armazenar presença
presentes = set()

# =====
# Configuracao do ArUco
# =====
dicionario = aruco.getPredefinedDictionary(aruco.DICT_4X4_50)
parametros = aruco.DetectorParameters()

# Inicializa a camera
cap = cv2.VideoCapture(0)

print("Pressione 'q' para encerrar e salvar a lista de presença.")
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Converter para escala de cinza
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detectar ArUcos
    cantos, ids, _ = aruco.detectMarkers(gray, dicionario, parameters=parametros)
```

```

if ids is not None:
    # Desenhar os marcadores detectados
    aruco.drawDetectedMarkers(frame, cantos, ids)
    for id in ids.flatten():
        if id in lista_chamada:
            presentes.add(lista_chamada[id])

# Mostrar imagem
cv2.imshow("Chamada com ArUco", frame)

# Sair ao apertar 'q'
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Liberar recursos
cap.release()
cv2.destroyAllWindows()

# =====
# Salvar presença em arquivo
# =====
with open("presenca.txt", "w", encoding="utf-8") as f:
    f.write("Lista de Presenca:\n\n")
    for nome in sorted(presentes, key=lambda x: remover_acentos(x).lower()):
        f.write(nome + "\n")

print("Presenca salva no arquivo presenca.txt")

```

Atenção!

Pode acontecer de o programa se confundir e identificar um ArUco mesmo sem ele estar sendo mostrado na câmera. Nesse caso, um aluno pode acabar sendo marcado como presente por engano.



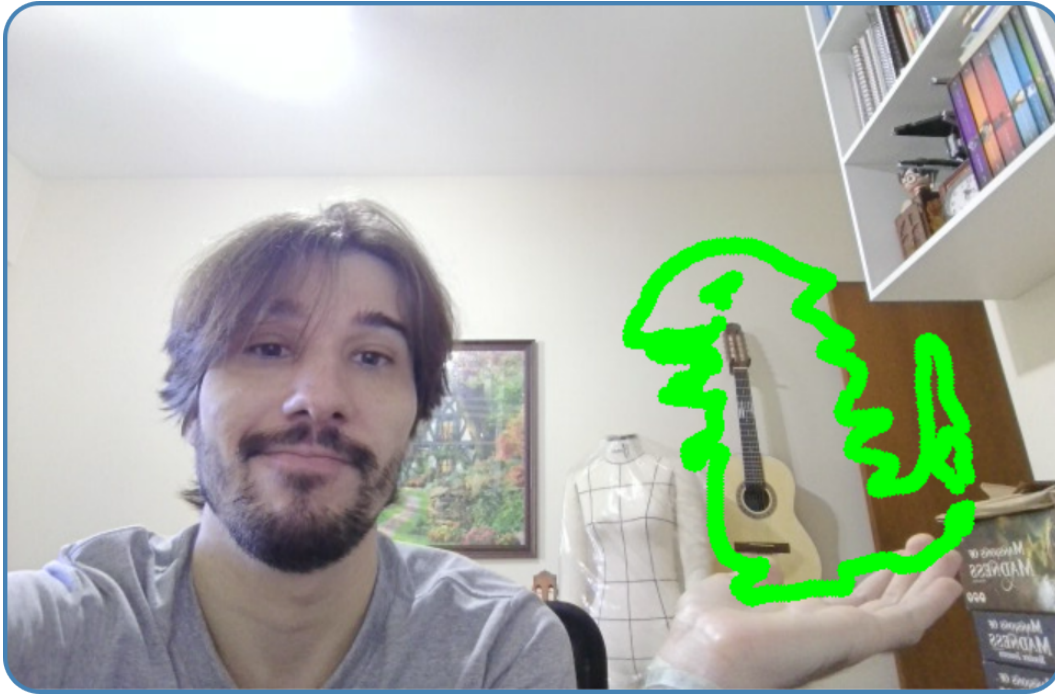
Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como ler os nomes dos alunos a partir de um arquivo de texto ou salvar a lista de presença em uma planilha do Excel.

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF01MA01 (Números)** — Reconhecimento de situações em que os números não indicam contagem nem ordem, mas funcionam como códigos de identificação. Nesta atividade, o ID do marcador ArUco atua justamente como um código de identificação de cada aluno, permitindo que o sistema registre a presença sem representar uma contagem sequencial.
- **EF01MA22 (Probabilidade e estatística)** — realização de pesquisas e organização de informações coletadas. A prática permite realizar enquetes utilizando o ID do marcador para organizar dados sobre os alunos, vinculando variáveis de interesse a cada código ArUco para exercitar a coleta e a sistematização de informações.



3.2 Desenhando com ArUcos



Vamos desenhar usando ArUcos! Neste exemplo, os marcadores 0, 1 e 2 serão usados para desenhar na tela com cores diferentes, enquanto o marcador 3 funcionará como uma borracha, permitindo apagar o que foi desenhado. Estamos usando o dicionário 4×4 .



O código da pintura pode ser encontrado neste [link](https://github.com/labvisio/kits-demonstracao-visao/blob/main/aruco_draw.py). Este exemplo foi adaptado do código do Lab Visio [8].

```
# Modificado de Lab Visio:
# https://github.com/labvisio/kits-demonstracao-visao/blob/main/aruco_draw.py
import cv2
import cv2.aruco as aruco

cap = cv2.VideoCapture(0)

# Lista de pontos desenhados: (x, y, cor)
desenho = []

# Dicionario ArUco
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_250)
params = aruco.DetectorParameters()

detector = aruco.ArucoDetector(aruco_dict, params)

while True:
    ret, frame = cap.read()
    if not ret:
        break
```

```

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
corners, ids, _ = detector.detectMarkers(gray)

if ids is not None:
    for i in range(len(ids)):
        id = ids[i][0]
        pts = corners[i][0]

        # Centro do marcador
        cx = int(pts[:, 0].mean())
        cy = int(pts[:, 1].mean())

        # Escolha da acao
        if id == 0:
            desenho.append((cx, cy, (0, 0, 255))) # vermelho
        elif id == 1:
            desenho.append((cx, cy, (0, 255, 0))) # verde
        elif id == 2:
            desenho.append((cx, cy, (255, 0, 0))) # azul
        elif id == 3:
            # Borracha: remove pontos proximos
            raio = 20
            desenho = [p for p in desenho if (p[0]-cx)**2 + (p[1]-cy)**2 > raio**2]

            # Desenha o circulo da borracha na tela
            cv2.circle(frame, (cx, cy), raio, (0, 255, 255), 2) # amarelo

# Desenhar tudo
for x, y, cor in desenho:
    cv2.circle(frame, (x, y), 5, cor, -1)

frame = cv2.flip(frame, 1) # 1 = horizontal (efeito espelho)
cv2.imshow("Pintura com ArUco", frame)

key = cv2.waitKey(1)
if key == ord('q'):#27: # ESC
    break
elif key == ord('l'):
    desenho = []

cap.release()
cv2.destroyAllWindows()

```

Do jeito que está, para limpar toda a janela basta pressionar a tecla “l” (L minúsculo) no teclado. Para encerrar o programa, pressione “q”.

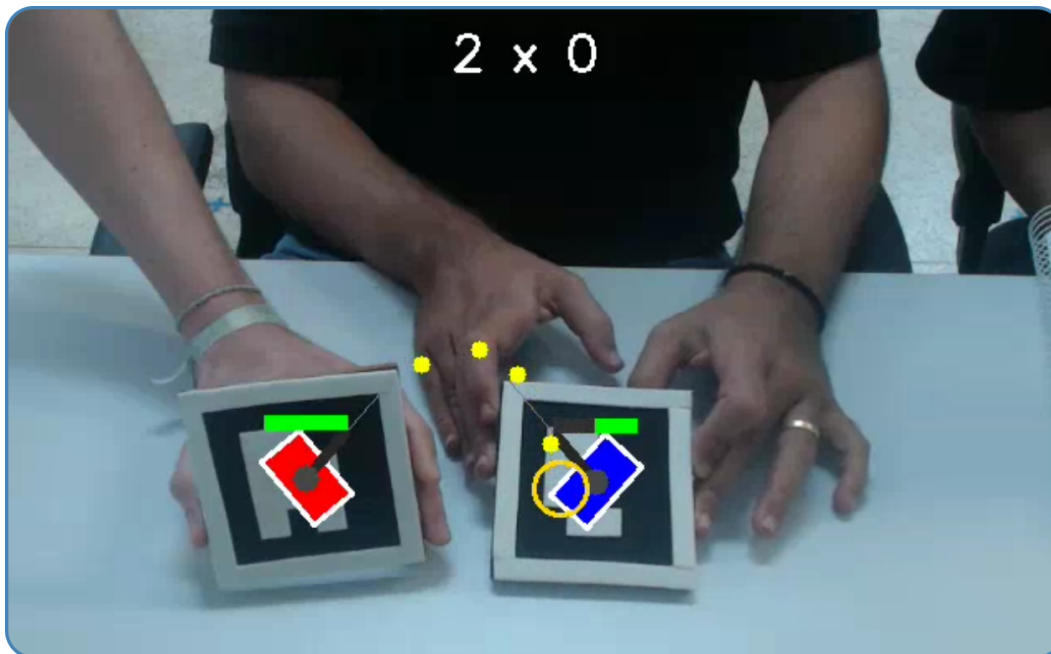
Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como mudar as cores utilizadas, adicionar novas cores associadas a outros IDs de ArUco, adicionar um modo de salvar o desenho como imagem, e implementar diferentes estilos de pintura, como marcador, spray ou tinta com difusão.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF15AR26 (Artes integradas)** — Exploração de tecnologias e recursos digitais nos processos de criação artística, com ênfase na aplicação de visão computacional, em que o estudante utiliza marcadores ArUco para controlar, em tempo real, cores e espessuras de traço diretamente na interface de desenho.
- **EF15AR02 (Artes Visuais)** — Reconhecimento e experimentação dos elementos constitutivos das artes visuais (ponto, linha, forma, cor, espaço, movimento), explorados por meio do deslocamento físico do marcador no espaço para gerar desenhos digitais em tempo real.
- **EF15AR04 (Artes Visuais)** — Experimentação de diferentes formas de expressão artística com apoio de recursos tecnológicos, incentivando o aluno a testar diversos estilos de pintura digital e ferramentas de edição integradas ao código.
- **EF01MA01 (Números)** — Reconhecimento de que os números naturais podem funcionar como códigos de identificação; na prática, os IDs dos marcadores (0, 1, 2) não indicam contagem ou ordem, mas acionam funções e cores específicas no sistema.
- **EF01MA11–EF01MA12 (Geometria)** — Desenvolvimento de noções de lateralidade e orientação espacial ao descrever a localização e o movimento do marcador no mundo físico (frente, atrás, esquerda, direita) para projetar o traço no plano virtual.
- **EF02MA12 (Geometria)** — Identificação e registro de localização e deslocamentos de objetos no espaço, indicando mudanças de direção e de sentido conforme o aluno movimentar o ArUco para criar formas geométricas ou figuras na tela.

3.3 Batalha de canhões com ArUcos



Vamos fazer uma batalha de canhões usando ArUcos! Neste exemplo, os marcadores 88 e 973 representam dois canhões que seguem sua posição na câmera. Ao girar o marcador, você controla a direção do disparo. Use “a” para atirar com o canhão vermelho, “l” (L minúsculo) para o azul e “r” para reiniciar a pontuação.

O código do canhão pode ser encontrado neste [link](#).



```
import cv2
import cv2.aruco as aruco
import numpy as np
import time
import math
import random

# =====
# CONFIGURACOES
# =====
ID_TANQUE_verm = 88
ID_TANQUE_azul = 973

GRAVIDADE = 500
COOLDOWN = 0.5
DANO = 25
HP_MAX = 100

cap = cv2.VideoCapture(0)
```

```

detector = aruco.ArucoDetector(
    aruco.getPredefinedDictionary(aruco.DICT_4X4_1000),
    aruco.DetectorParameters()
)

# =====
# ESTADO DO JOGO
# =====
tanques = {"verm": None, "azul": None}
projeteis = []
explosoes = []
placar = {"verm": 0, "azul": 0}

vento = random.uniform(-200, 200)
vencedor = None
tempo_vitoria = 2

# =====
# CRIACAO DE OBJETOS
# =====
def criar_tanque(x, y, angulo):
    return {
        "x": x,
        "y": y,
        "ang": angulo,
        "hp": HP_MAX,
        "ultimo_tiro": 0
    }

def criar_projetil(tanque, dono):
    ang = tanque["ang"]
    v = 300

    return {
        "x": tanque["x"] + 35 * math.cos(ang),
        "y": tanque["y"] - 35 * math.sin(ang),
        "vx": v * math.cos(ang),
        "vy": -v * math.sin(ang),
        "tempo": time.time(),
        "dono": dono
    }

def criar_explosao(x, y):
    return {"x": x, "y": y, "tempo": time.time()}

# =====
# DESENHO
# =====
def desenhar_tanque(frame, t, cor):
    x, y, ang = int(t["x"]), int(t["y"]), t["ang"]

```



```

# corpo
corpo = np.array([[-25, -15], [25, -15], [25, 15], [-25, 15]])
R = np.array([[math.cos(ang), -math.sin(ang)],
              [math.sin(ang),  math.cos(ang)]])

pts = np.dot(corpo, R.T) + [x, y]
pts = pts.astype(int)

cv2.fillPoly(frame, [pts], cor)
cv2.polylines(frame, [pts], True, (255, 255, 255), 2)

# canhao
ponta = (int(x + 35 * math.cos(ang)),
         int(y - 35 * math.sin(ang)))
cv2.line(frame, (x, y), ponta, (40, 40, 40), 6)

# barra de vida
hp = int(50 * max(t["hp"], 0) / HP_MAX)
cv2.rectangle(frame, (x-25, y-38), (x+25, y-30), (50,50,50), -1)
cv2.rectangle(frame, (x-25, y-38), (x-25+hp, y-30), (0,255,0), -1)

# =====
# DETECCAO DOS ARUCOS
# =====
def atualizar_tanques(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = detector.detectMarkers(gray)

    if ids is None:
        return

    for i, marker_id in enumerate(ids.flatten()):
        pts = corners[i][0]

        cx = pts[:, 0].mean()
        cy = pts[:, 1].mean()

        dx = pts[0][0] - cx
        dy = cy - pts[0][1]
        ang = math.atan2(dy, dx)

        if marker_id == ID_TANQUE_verm:
            lado = "verm"
        elif marker_id == ID_TANQUE_azul:
            lado = "azul"
        else:
            continue

        if tanques[lado] is None:
            tanques[lado] = criar_tanque(cx, cy, ang)
        else:
            tanques[lado].update(x=cx, y=cy, ang=ang)

```

```

# =====
# FISICA DOS PROJETEIS
# =====
def atualizar_projeteis(frame, largura, altura):
    global vencedor, tempo_vitoria

    novos = []

    for p in projeteis:
        t = time.time() - p["tempo"]

        if t < 0.2:
            novos.append(p)
            continue

        x = int(p["x"] + p["vx"]*t + 0.5*vento*t*t)
        y = int(p["y"] + p["vy"]*t + 0.5*GRAVIDADE*t*t)

        if 0 < x < largura and 0 < y < altura:
            cv2.circle(frame, (x, y), 5, (0, 255, 255), -1)

            # colisao com tanques
            for lado in ["verm", "azul"]:
                tnk = tanques[lado]

                if not tnk or tnk["hp"] <= 0 or p["dono"] == lado:
                    continue

                if (x - tnk["x"])**2 + (y - tnk["y"])**2 < 25**2:
                    tnk["hp"] -= DANO
                    explosoes.append(criar_explosao(x, y))

                    if tnk["hp"] <= 0:
                        outro = "azul" if lado == "verm" else "verm"
                        placar[outro] += 1
                        vencedor = outro
                        tempo_vitoria = time.time()

                    break
            else:
                novos.append(p)

    return novos

# =====
# EXPLOSOES
# =====
def atualizar_explosoes(frame):
    novas = []

    for e in explosoes:

```

```

    t = time.time() - e["tempo"]

    if t < 0.5:
        raio = int(10 + 40*t)
        cor = (0, int(255*(1-t)), 255)
        cv2.circle(frame, (int(e["x"]), int(e["y"])), raio, cor, 2)
        novas.append(e)

    return novas

# =====
# LOOP PRINCIPAL
# =====
while True:
    ret, frame = cap.read()
    if not ret:
        break

    h, w = frame.shape[:2]

    atualizar_tanques(frame)

    # desenhar tanques
    if tanques["verm"] and tanques["verm"]["hp"] > 0:
        desenhar_tanque(frame, tanques["verm"], (0, 0, 255))

    if tanques["azul"] and tanques["azul"]["hp"] > 0:
        desenhar_tanque(frame, tanques["azul"], (255, 0, 0))

    # atualizar jogo
    projeteis = atualizar_projeteis(frame, w, h)
    explosoes = atualizar_explosoes(frame)

    frame = cv2.flip(frame, 1)

    # HUD
    cv2.putText(frame, f"{placar['verm']} x {placar['azul']}",
                (w//2 - 50, 40), 0, 1, (255,255,255), 2)

    cv2.putText(frame, f"Vento: {int(vento)}",
                (20, h-20), 0, 0.7, (200,200,200), 2)

    # reset apos vitoria
    if vencedor:
        if vencedor == "verm":
            cv2.putText(frame, "Vermelho ganhou!",
                        (w//2 - 140, 80), cv2.FONT_HERSHEY_SIMPLEX,
                        1, (0, 255, 0), 3)
        else:
            cv2.putText(frame, "Azul ganhou!",
                        (w//2 - 100, 80), cv2.FONT_HERSHEY_SIMPLEX,
                        1, (0, 255, 0), 3)
        if time.time() - tempo_vitoria > 2:

```

```

projeteis.clear()
vento = random.uniform(-200, 200)
vencedor = None

for t in tanques.values():
    if t:
        t["hp"] = HP_MAX

cv2.imshow("Batalha de Tanques", frame)

# =====
# CONTROLES
# =====
key = cv2.waitKey(1)

if key == ord('q'):
    break

if key == ord('a') and tanques["verm"]:
    t = tanques["verm"]
    if time.time() - t["ultimo_tiro"] > COOLDOWN:
        projeteis.append(criar_projetil(t, "verm"))
        t["ultimo_tiro"] = time.time()

if key == ord('l') and tanques["azul"]:
    t = tanques["azul"]
    if time.time() - t["ultimo_tiro"] > COOLDOWN:
        projeteis.append(criar_projetil(t, "azul"))
        t["ultimo_tiro"] = time.time()

if key == ord('r'):
    placar = {"verm": 0, "azul": 0}
    projeteis.clear()
    explosoes.clear()
    vento = random.uniform(-200, 200)

    for t in tanques.values():
        if t:
            t["hp"] = HP_MAX

cap.release()
cv2.destroyAllWindows()

```

Para encerrar o programa, pressione “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como mudar as cores dos canhões, ajustar a velocidade dos projéteis ou modificar a gravidade para observar diferentes trajetórias.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF05MA15 (Geometria)** — Interpretação e representação da movimentação de objetos no plano cartesiano, indicando mudanças de direção, sentido e giros. Na atividade, a rotação física do marcador ArUco é traduzida pelo código em ângulos de inclinação do canhão, permitindo ao aluno visualizar como diferentes graus de giro alteram a trajetória parabólica do projétil.
- **EF01MA12 (Matemática)** — Descrição da localização de objetos no espaço segundo um dado ponto de referência. A dinâmica de jogo exige que o estudante identifique a posição do seu disparo e do alvo (o oponente) utilizando referenciais externos explícitos, desenvolvendo a noção de coordenadas e distância entre pontos no espaço virtual.
- **EF12EF05 (Educação Física)** — Experimentação e fruição de atividades que consistem em lançar um objeto procurando acertar um alvo específico. A batalha simula a lógica interna dos "Esportes de Precisão", exigindo que o aluno desenvolva estratégias de controle e ajuste de força e direção para atingir o objetivo, integrando a coordenação motora fina ao pensamento estratégico.
- **EI02ET02 (Espaços, Tempos, Quantidades, Relações e Transformações)** — Observação e descrição de fenômenos naturais, como o vento e a gravidade. A presença das variáveis *vento* (aleatória) e *gravidade* (constante) no código Python permite ao professor demonstrar como forças externas influenciam o movimento dos corpos, transformando o jogo em um laboratório de experimentação física.

3.4 Museu de arte com ArUcos



Vamos substituir os ArUcos vistos pela câmera por quadros de arte! Neste exemplo, você pode escolher quais marcadores serão trocados por imagens e definir o tamanho em que essas artes aparecerão na tela.



O código do museu pode ser encontrado neste [link](#). Já as três artes usadas podem ser acessadas [aqui](#).

```
import cv2
import cv2.aruco as aruco
import numpy as np

cap = cv2.VideoCapture(0)

# Configuracao: ID -> (caminho PNG, escala)
imagens = {
    0: ("mona.png", 10.0),
    1: ("grito.png", 5),
    2: ("noite.png", 6)
}

# carregar imagens (com alpha)
imagens_carregadas = {}
for k, (path, escala) in imagens.items():
    img = cv2.imread(path, cv2.IMREAD_UNCHANGED) # mantem transparencia
    imagens_carregadas[k] = (img, escala)
```

```
# ArUco
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_250)
params = aruco.DetectorParameters()
detector = aruco.ArUcoDetector(aruco_dict, params)

def overlay_png(frame, png, corners, escala):
    h_img, w_img = png.shape[:2]

    # dimensoes do ArUco (aproximadas)
    largura_aruco = np.linalg.norm(corners[0] - corners[1])
    altura_aruco = np.linalg.norm(corners[1] - corners[2])

    # aplicar escala no alvo
    largura_aruco *= escala
    altura_aruco *= escala

    # manter proporcao (fit dentro do ArUco)
    fator = min(largura_aruco / w_img, altura_aruco / h_img)

    new_w = int(w_img * fator)
    new_h = int(h_img * fator)

    png = cv2.resize(png, (new_w, new_h))

    # centralizar dentro do ArUco
    cx = np.mean(corners[:, 0])
    cy = np.mean(corners[:, 1])

    pts_dst = np.array([
        [cx - new_w/2, cy - new_h/2],
        [cx + new_w/2, cy - new_h/2],
        [cx + new_w/2, cy + new_h/2],
        [cx - new_w/2, cy + new_h/2]
    ], dtype=np.float32)

    pts_src = np.array([
        [0, 0],
        [new_w, 0],
        [new_w, new_h],
        [0, new_h]
    ], dtype=np.float32)

    # homografia
    M, _ = cv2.findHomography(pts_src, pts_dst)

    # alpha
    if png.shape[2] == 4:
        rgb = png[:, :, :3]
        alpha = png[:, :, 3] / 255.0
    else:
        rgb = png
        alpha = np.ones((new_h, new_w))
```

```

warped = cv2.warpPerspective(rgb, M, (frame.shape[1], frame.shape[0]))
mask = cv2.warpPerspective(alpha, M, (frame.shape[1], frame.shape[0]))

for c in range(3):
    frame[:, :, c] = frame[:, :, c] * (1 - mask) + warped[:, :, c] * mask

return frame

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = detector.detectMarkers(gray)

    if ids is not None:
        for i in range(len(ids)):
            id = ids[i][0]

            if id in imagens_carregadas:
                png, escala = imagens_carregadas[id]
                frame = overlay_png(frame, png, corners[i][0], escala)

    frame = cv2.flip(frame, 1)
    cv2.imshow("Substituicao ArUco", frame)

    if cv2.waitKey(1) == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()

```

Veja que para adicionar novas artes, mudamos essas linhas de código:

```

imagens = {
    0: ("mona.png", 10.0),
    1: ("grito.png", 5),
    2: ("noite.png", 6)
}

```

No exemplo, a imagem da Mona Lisa é exibida com uma escala de 10 vezes quando o marcador 0 é detectado. É possível adicionar quantas artes desejar, bastando incluir novas entradas (linhas) separadas por vírgula (exceto na última). Abaixo, adicionamos outra arte:

```

imagens = {
    0: ("mona.png", 10.0),
    1: ("grito.png", 5),
    2: ("noite.png", 6),
    15: ("vangogh.png", 8)
}

```

Para encerrar o programa, pressione a tecla “q”.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF01MA01 (Números)** — Reconhecimento de números como códigos de identificação. Assim como na atividade da chamada, o aluno consolida a ideia de que o ID do marcador não indica uma quantidade de quadros, mas sim um código que “chama” uma obra específica no banco de dados do programa.
- **EF15AR01 (Artes Visuais)** — Identificação e apreciação de formas distintas das artes tradicionais e contemporâneas. Ao substituir os marcadores ArUco por obras clássicas, a atividade cultiva a percepção e o imaginário, ampliando o repertório imagético do estudante através da interação tecnológica.
- **EF15AR07 (Artes Visuais)** — Reconhecimento de categorias do sistema das artes, como o museu e a galeria. A prática permite que o aluno experimente a organização de uma galeria digital no ambiente escolar, compreendendo o papel dos espaços de exposição e curadoria.
- **EF05HI10 (Registros da História)** — Inventário dos patrimônios materiais da humanidade e análise de suas mudanças e permanências ao longo do tempo. A criação do acervo digital no museu escolar permite discutir o que torna uma obra (como a Mona Lisa) um patrimônio cultural e como a tecnologia auxilia na sua preservação e divulgação.
- **EF04ER05 (Ensino Religioso)** — Identificação de representações religiosas em diferentes expressões artísticas. Caso o acervo escolhido inclua pinturas sacras ou ícones de diferentes tradições, a atividade torna-se um recurso para reconhecer a arte como parte da identidade de diversas culturas e crenças.
- **EM13LP11** — Realização de curadoria de informações para projetos discursivos e artísticos. No Ensino Médio, a proposta incentiva o estudante a atuar como um “leitor-fruidor”, exercendo o protagonismo ao selecionar obras e desvendar camadas de sentido em um acervo virtual.

3.5 Museu de arte com perspectiva e ArUcos



No exemplo anterior, as obras de arte apareciam sobre os ArUcos, mas continuavam sempre voltadas para a câmera. Agora, vamos melhorar esse efeito fazendo com que as imagens acompanhem a inclinação e a perspectiva do marcador detectado.

Para isso, utilizaremos uma técnica chamada homografia, que permite deformar a imagem para que ela se encaixe corretamente sobre o ArUco. Assim, quando o marcador é girado ou inclinado, a obra também acompanha esse movimento, deixando o efeito de Realidade Aumentada muito mais natural.



O código do museu com perspectiva pode ser encontrado neste [link](#). Já as três artes usadas podem ser acessadas [aqui](#).

```
import cv2
import cv2.aruco as aruco
import numpy as np

cap = cv2.VideoCapture(0)

# Configuracao: ID -> (caminho PNG, escala)
imagens = {
    0: ("mona.png", 2.0),
    1: ("grito.png", 2.0),
    2: ("noite.png", 2.0)
}

# carregar imagens (com alpha)
imagens_carregadas = {}
for k, (path, escala) in imagens.items():
    img = cv2.imread(path, cv2.IMREAD_UNCHANGED) # mantem transparencia
```

```

    imagens_carregadas[k] = (img, escala)

# ArUco
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_250)
params = aruco.DetectorParameters()
detector = aruco.ArucoDetector(aruco_dict, params)

def overlay_png(frame, png, corners, escala=1.0):

    h_img, w_img = png.shape[:2]
    h_img2, w_img2 = h_img/2, w_img/2

    # =====
    # CENTRO DO PNG COM QUADRADO
    # =====
    centro_src = np.array([w_img2, h_img2], dtype=np.float32)
    d_png = min(w_img2, h_img2)/escala
    pts_src = np.array([
        [centro_src[0]+d_png,centro_src[1]-d_png],
        [centro_src[0]-d_png,centro_src[1]-d_png],
        [centro_src[0]-d_png,centro_src[1]+d_png],
        [centro_src[0]+d_png,centro_src[1]+d_png]
    ], dtype=np.float32)

    # =====
    # ARUCO QUADRADO
    # =====
    pts_dst = np.array(corners, dtype=np.float32)

    M = cv2.getPerspectiveTransform(pts_src, pts_dst)

    # =====
    # IMAGEM
    # =====
    if png.shape[2] == 4:
        rgb = png[:, :, :3]
        alpha = png[:, :, 3] / 255.0
    else:
        rgb = png
        alpha = np.ones((h_img, w_img))

    warped = cv2.warpPerspective(rgb, M, (frame.shape[1], frame.shape[0]))
    mask = cv2.warpPerspective(alpha, M, (frame.shape[1], frame.shape[0]))

    for c in range(3):
        frame[:, :, c] = frame[:, :, c] * (1 - mask) + warped[:, :, c] * mask

    return frame

while True:
    ret, frame = cap.read()
    if not ret:
        break

```

```

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
corners, ids, _ = detector.detectMarkers(gray)

if ids is not None:
    for i in range(len(ids)):
        id = ids[i][0]

        if id in imagens_carregadas:
            png, escala = imagens_carregadas[id]
            frame = overlay_png(frame, png, corners[i][0], escala)

frame = cv2.flip(frame, 1)
cv2.imshow("Substituicao ArUco", frame)

if cv2.waitKey(1) == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Veja que para adicionar novas artes, mudamos essas linhas de código, assim como o exemplo anterior:

```

imagens = {
    0: ("mona.png", 2.0),
    1: ("grito.png", 5),
    2: ("noite.png", 6)
}

```

No exemplo, a imagem da Mona Lisa é exibida com uma escala de 2 vezes quando o marcador 0 é detectado. É possível adicionar quantas artes desejar, bastando incluir novas entradas (linhas) separadas por vírgula (exceto na última). Abaixo, adicionamos outra arte:

```

imagens = {
    0: ("mona.png", 2.0),
    1: ("grito.png", 5),
    2: ("noite.png", 6),
    15: ("vangogh.png", 2.0)
}

```

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como usar uma foto ou uma imagem do computador com ArUcos no lugar de usar a câmera (em vez de captura ao vivo), e trocar as imagens exibidas pelos ArUcos a qualquer momento, substituindo as obras de arte por outras imagens quaisquer. Outra ideia é usar desenhos feitos pelos próprios alunos, criando um “museu da turma”, onde cada ArUco mostra uma produção diferente.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF15AR01 (Artes Visuais)** — Identificação e apreciação de formas distintas das artes tradicionais e contemporâneas. Ao substituir os marcadores ArUco por obras clássicas, a atividade cultiva a percepção e o imaginário, ampliando o repertório imagético do estudante através da interação tecnológica.
- **EF15AR07 (Artes Visuais)** — reconhecimento de categorias do sistema das artes, como o museu e a galeria. A prática permite que o aluno experimente a organização de uma galeria digital no ambiente escolar, compreendendo o papel dos espaços de exposição e curadoria.
- **EF15AR26 (Artes Integradas)** — exploração de tecnologias e recursos digitais nos processos de criação artística. A atividade aprofunda o uso da tecnologia na arte ao utilizar a técnica de homografia para deformar a imagem de acordo com a perspectiva do marcador, criando uma experiência imersiva de Realidade Aumentada.
- **EF05MA15 (Geometria)** — Interpretação, descrição e representação da localização ou movimentação de objetos no plano cartesiano, indicando mudanças de direção, sentido e giros. No nível básico, a atividade ensina como o sistema identifica o “giro” do marcador (eixo Z) e altera a orientação da imagem projetada.
- **EF07MA21 (Geometria)** — Reconhecimento e construção de figuras obtidas por simetrias de translação, rotação e reflexão, utilizando softwares de geometria dinâmica e vinculando o estudo a representações de obras de arte. O uso do Python como ferramenta de geometria dinâmica permite ao aluno perceber a rotação como uma variável matemática que altera a posição da obra na tela.
- **EF09MA17 (Geometria)** — Reconhecimento de vistas ortogonais de figuras espaciais e aplicação desse conhecimento para desenhar objetos em perspectiva. A atividade inverte o processo tradicional: em vez de o aluno desenhar no papel, ele utiliza a visão computacional para ver como a máquina projeta imagens em perspectiva sobre um plano inclinado.
- **EM13MAT105** — Utilização de noções de transformações isométricas (translação, reflexão, rotação e composições destas) e transformações homotéticas para construir figuras e analisar elementos da natureza e diferentes produções humanas, como as obras de arte. Esta habilidade é o pilar central da atividade, pois o código utiliza a rotação e a inclinação do ArUco para deformar a imagem de forma correspondente.
- **EM13MAT509** — Investigação da deformação de ângulos e áreas provocada pelas diferentes projeções, utilizando suporte de tecnologia digital. Embora o código original mencione cartografia, o conceito matemático de observar como uma projeção deforma uma imagem plana (o marcador) ao ser visualizada de diferentes ângulos é exatamente o que o algoritmo de homografia executa na prática.

3.6 Jogo da memória com ArUcos



Vamos criar um jogo da memória utilizando ArUcos. Neste exemplo, os marcadores são substituídos por imagens, formando as cartas do jogo. Ao identificar e combinar corretamente os pares, o jogador avança até completar todos, vencendo a partida. A atividade pode ser explorada em sala de aula para trabalhar atenção, memória e raciocínio lógico de forma lúdica, além de introduzir conceitos básicos de visão computacional.



O código do jogo da memória pode ser encontrado neste [link](#). Já as cartas usadas podem ser acessadas [aqui](#).

```
import cv2
import cv2.aruco as aruco
import numpy as np
import time

# =====
# CONFIGURACAO
# =====
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_1000)
detector = aruco.ArucoDetector(aruco_dict, aruco.DetectorParameters())

pares = {
    973: "mona.png", 62: "mona.png",
    118: "noite.png", 88: "noite.png",
    49: "grito.png", 17: "grito.png"
}

imagens = {k: cv2.imread(v) for k, v in pares.items()}
```

```

fixos = set()

# controle de tentativas (varios pares ao mesmo tempo)
tentativas = []
# cada tentativa: {"ids": [id1, id2], "tempo": t}

# =====
# LOOP PRINCIPAL
# =====
while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    corners, ids, _ = detector.detectMarkers(gray)

    detectados = []
    if ids is not None:
        for i, marker_id in enumerate(ids.flatten()):
            if marker_id in pares and marker_id not in fixos:
                detectados.append(marker_id)

    # =====
    # CRIAR NOVAS TENTATIVAS
    # =====
    usados = set()
    for i in range(len(detectados)):
        for j in range(i+1, len(detectados)):
            id1, id2 = detectados[i], detectados[j]

            if id1 in usados or id2 in usados:
                continue

            # evita duplicar tentativa
            ja_existe = any(set(t["ids"]) == {id1, id2} for t in tentativas)

            if not ja_existe:
                tentativas.append({
                    "ids": [id1, id2],
                    "tempo": time.time()
                })
                usados.update([id1, id2])

    # =====
    # PROCESSAR TENTATIVAS
    # =====
    novas_tentativas = []
    revelados = set()

    for t in tentativas:
        id1, id2 = t["ids"]

```

```

    if pares[id1] == pares[id2]:
        fixos.update([id1, id2])
    elif time.time() - t["tempo"] < 2:
        novas_tentativas.append(t)
        revelados.update([id1, id2])

tentativas = novas_tentativas
# =====
# DESENHO
# =====
if ids is not None:
    for i, marker_id in enumerate(ids.flatten()):
        pts = corners[i][0].astype(int)

        cx = int(pts[:, 0].mean())
        cy = int(pts[:, 1].mean())

        if marker_id in fixos or marker_id in revelados:
            marker_id = int(marker_id)
            img = imagens[marker_id]
            if img is not None:
                img = cv2.resize(img, (100, 100))
                x1, y1 = cx - 50, cy - 50
                x2, y2 = cx + 50, cy + 50

                if 0 < x1 and 0 < y1 and x2 < frame.shape[1] and y2 < frame.shape[0]:
                    frame[y1:y2, x1:x2] = img
            else:
                cv2.polylines(frame, [pts], True, (0, 0, 255), 2)

frame = cv2.flip(frame, 1)
# =====
# VITORIA
# =====
if len(fixos) == len(pares):
    cv2.putText(frame, "Voce venceu!", (50, 50),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.namedWindow("Jogo da Memoria com ArUco", cv2.WINDOW_NORMAL)
cv2.imshow("Jogo da Memoria com ArUco", frame)
if cv2.waitKey(1) == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()

```

Veja que para adicionar novas cartas, mudamos essas linhas de código, em que cada linha mostra um par (IDs e nome da imagem):

```

pares = {
    973: "mona.png", 62: "mona.png",
    118: "noite.png", 88: "noite.png",
    49: "grito.png", 17: "grito.png"
}

```


No exemplo, temos três pares possíveis. Se quiser adicionar mais um par com os ArUcos 10 e 15 para a imagem da Santa Ceia:

```
pares = {  
    973: "mona.png", 62: "mona.png",  
    118: "noite.png", 88: "noite.png",  
    49: "grito.png", 17: "grito.png",  
    10: "ceia.jpg", 15: "ceia.jpg"  
}
```

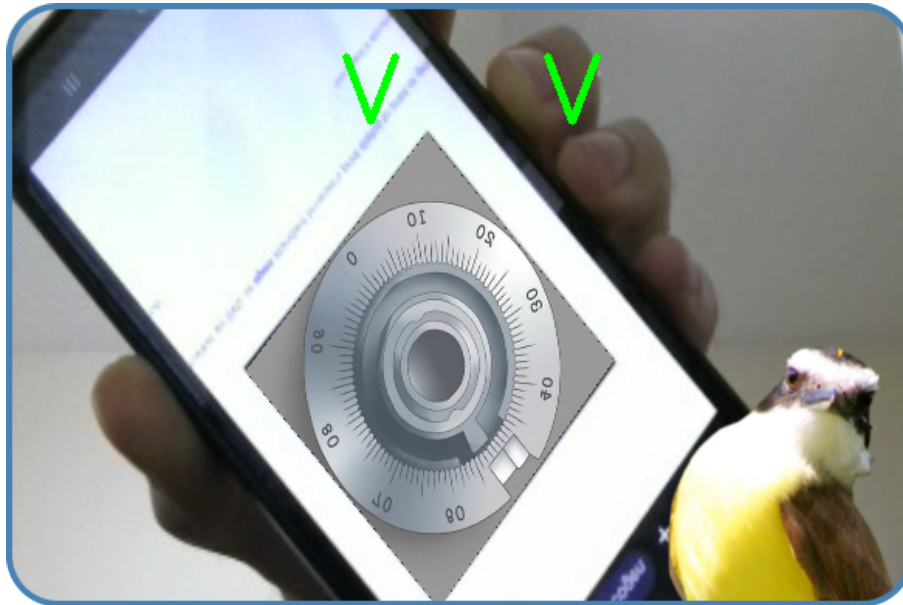
Para encerrar o programa, pressione a tecla “q”.

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF01MA02 (Números)** — Contagem exata ou aproximada utilizando estratégias como o pareamento. No jogo, essa habilidade é transposta para o ambiente digital, exigindo que o aluno realize a correspondência direta entre os IDs dos marcadores ArUco e as imagens que eles representam para formar os pares corretamente.
- **EF01MA10 (Álgebra)** — Descrição de elementos ausentes em sequências recursivas. A atividade introduz noções de reconhecimento de padrões, fundamentais para esta unidade temática, ao exigir a identificação de repetições lógicas para a conclusão do jogo.
- **Competência Geral 2 (Pensamento Científico, Crítico e Criativo):** estímulo à curiosidade intelectual e ao pensamento crítico. A dinâmica desafia o estudante a criar estratégias e refletir sobre suas escolhas para resolver o problema proposto.



3.7 Abrindo um cofre com ArUcos



Nesta atividade, vamos desenvolver um jogo que simula a abertura de um cofre utilizando marcadores ArUco. Um marcador específico (no código apresentado, o de ID 8) será substituído por uma imagem de um disco de combinação, semelhante ao utilizado em cofres mecânicos.

O objetivo do jogador é girar o ArUco para encontrar a combinação correta. Para isso, será necessário alternar a direção de rotação a cada etapa, reproduzindo o funcionamento de cofres tradicionais, nos quais a sequência de giros deve ser seguida com precisão para que a abertura seja realizada com sucesso.



O código do jogo do cofre pode ser encontrado neste [link](#). Já as imagens e sons usados podem ser acessados [aqui](#).

```
import cv2, numpy as np, pygame

# ID do ArUco =====
ID_COFRE = 8

# Combinacao =====
seq_ang = [90, 180, 270, 0]
seq_dir = [(-1)**i for i in range(len(seq_ang))]

# =====
# ARQUIVOS
# =====
SND_OPEN = './cofre/abriu.mp3'
SND_CLICK = './cofre/click.mp3'
SND_RIGHT = './cofre/right_angle.mp3'
```

```

IMG_LOCK = './cofre/combinacao.png'
IMG_FINAL = './cofre/seven.png'

# =====
# FUNCOES
# =====
def angulo(p1, p2):
    ang = np.degrees(np.arctan2(p2[1]-p1[1], p2[0]-p1[0]))
    return ang + 360 if ang < 0 else ang

def subangulo(angvelho, angnovo):
    distancia = (angnovo - angvelho + 180) % 360 - 180
    return distancia

def direcao(delta, dir, limiar):
    if abs(delta) > 1:
        if (np.sign(delta) != dir and abs(delta) > limiar) or (np.sign(delta) == dir):
            return np.sign(delta)
    return 0

# =====
# MAIN
# =====
pygame.init(); pygame.mixer.init()
pygame.mixer.set_num_channels(8)

ch_click = pygame.mixer.Channel(0)
ch_right = pygame.mixer.Channel(1)
ch_open = pygame.mixer.Channel(2)

click = pygame.mixer.Sound(SND_CLICK)
open_s = pygame.mixer.Sound(SND_OPEN)
right = pygame.mixer.Sound(SND_RIGHT)

lock = cv2.imread(IMG_LOCK)
final = cv2.resize(cv2.imread(IMG_FINAL), (900, 900))

h, w = lock.shape[:2]
src = np.float32([[0,0],[w,0],[w,h],[0,h]])
cap = cv2.VideoCapture(0)

aruco = cv2.aruco.ArucoDetector(cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50),
                                cv2.aruco.DetectorParameters())

LIM = 8
state = 0
last_ang = 0
mostrar_final = False

# =====
# LOOP
# =====
while True:
    ret, frame = cap.read()

```

```

if not ret: break

frame = cv2.resize(frame, (900,900))
out = frame.copy()

corners, ids, _ = aruco.detectMarkers(frame)

if ids is not None:
    for pts, i in zip(corners, ids):
        if i == ID_COFRE:
            pts = pts[0]
            ang = angulo(pts[0], pts[1])
            delta = subangulo(last_ang,ang)
            d = direcao(delta,seq_dir[state],LIM)

            # verifica direcao
            if d != 0 and mostrar_final != True:
                last_ang = ang
                click.play()
                if d != seq_dir[state]:
                    state = 0
                    print("Errou!")
                    cv2.putText(out, "X", (450, 100), cv2.FONT_HERSHEY_SIMPLEX, 2,
                                (0, 0, 255), 3)
                else:
                    # verifica angulo
                    if abs(ang - seq_ang[state]) < 6:
                        if state == 3:
                            ch_open.play(open_s)
                            mostrar_final = True
                            print("Abriu!")
                        else:
                            ch_right.play(right)
                            print("Proximo!")
                            state += 1

            # overlay imagem no marcador
            H, _ = cv2.findHomography(src, pts)
            warp = cv2.warpPerspective(lock, H, (900,900))

            mask = cv2.cvtColor(warp, cv2.COLOR_BGR2GRAY)
            _, mask = cv2.threshold(mask,10,255,cv2.THRESH_BINARY)
            mask_inv = cv2.bitwise_not(mask)

            bg = cv2.bitwise_and(out,out,mask=mask_inv)
            fg = cv2.bitwise_and(warp,warp,mask=mask)

            out = cv2.add(bg, fg)
            if state == 1:
                cv2.putText(out, "V", (450, 100), cv2.FONT_HERSHEY_SIMPLEX, 2,
                            (0, 255, 0), 3)
            elif state == 2:
                cv2.putText(out, "V", (530, 100), cv2.FONT_HERSHEY_SIMPLEX, 2,
                            (0, 255, 0), 3)

```

```

        cv2.putText(out,"V",(400, 100),cv2.FONT_HERSHEY_SIMPLEX, 2,
        (0, 255, 0), 3)
    elif state == 3:
        cv2.putText(out,"V",(580, 100),cv2.FONT_HERSHEY_SIMPLEX, 2,
        (0, 255, 0), 3)
        cv2.putText(out,"V",(320, 100),cv2.FONT_HERSHEY_SIMPLEX, 2,
        (0, 255, 0), 3)
        cv2.putText(out,"V",(450, 100),cv2.FONT_HERSHEY_SIMPLEX, 2,
        (0, 255, 0), 3)

    # tela final
    if mostrar_final:
        cv2.imshow("Cofre", final)
    else:
        cv2.imshow("Cofre", cv2.flip(out,1))

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
pygame.mixer.quit()

```

Veja que, para alterar o ArUco utilizado e a combinação do cofre (inclusive aumentando o tamanho da sequência), basta modificar as seguintes linhas de código:

```

# ID do ArUco =====
ID_COFRE = 8

# Combinacao =====
seq_ang = [90,180,270,0]

```

Nas linhas acima, o marcador ArUco utilizado possui ID igual a 8. Para abrir o cofre, o jogador deve girar o marcador seguindo a sequência de ângulos 90°, 180°, 270° e 0°, na ordem correta.

Além disso, é possível personalizar as imagens e os áudios do projeto. Para isso, você pode substituir os arquivos da pasta por outros com o mesmo nome ou adicionar novos arquivos e modificar seus nomes no trecho de código apresentado a seguir:

```

# =====
# ARQUIVOS
# =====
SND_OPEN   = './cofre/abriu.mp3'           # Som de abertura do cofre
SND_CLICK  = './cofre/click.mp3'          # Barulho do disco de combinacao
SND_RIGHT  = './cofre/right_angle.mp3'    # Som do Bem-te-vi

IMG_LOCK   = './cofre/combinacao.png'     # Imagem do disco de combinacao
IMG_FINAL  = './cofre/seven.png'          # Imagem final do 7 (senha guardada no cofre)

```

Inclusive, caso não deseje utilizar o som do Bem-te-vi no jogo (o que é pouco provável), basta comentar a linha a seguir colocando um # na frente:

```

#ch_right.play(right)

```

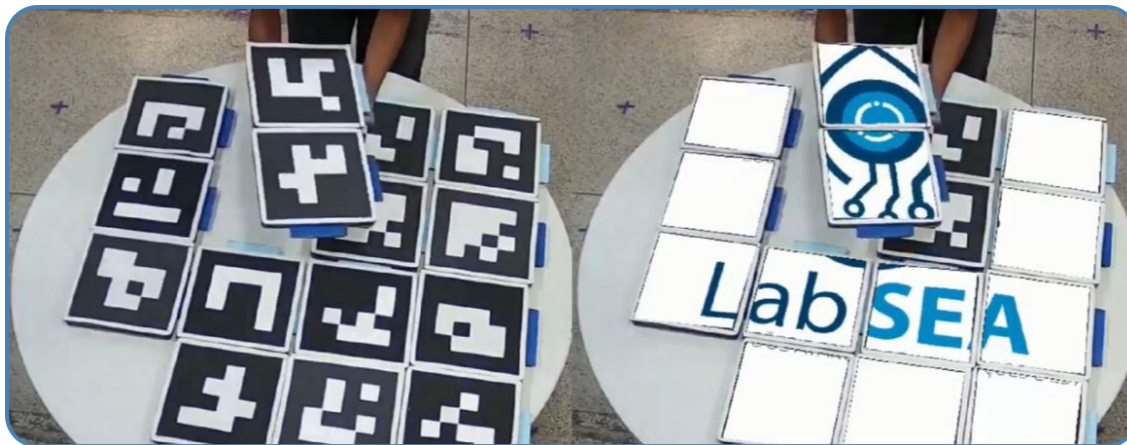
Para encerrar o programa, pressione a tecla “q”.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF05MA15 (Geometria)** — Interpretação e representação de localização e movimentação no espaço, incluindo mudanças de direção, sentido e giros. A atividade explora a relação entre a rotação física do marcador ArUco e o movimento virtual do disco de combinação.
- **EF02MA12 (Geometria)** — Identificação de mudanças de direção e sentido para atingir um objetivo específico, neste caso, encontrar a combinação correta do jogo.

3.8 Quebra-cabeça com ArUcos



Desta vez, vamos construir um quebra-cabeça interativo utilizando marcadores ArUco. A proposta é inspirada no clássico 15 Puzzle, no qual há um conjunto de 15 peças organizadas em uma grade 4×4, com uma posição vazia que permite o deslocamento das demais.

Serão usados 15 marcadores com IDs de 1 a 15, enquanto a décima sexta posição será o espaço vazio. Cada marcador representará um fragmento de uma imagem e, quando organizados corretamente, formarão a figura completa. O objetivo dos jogadores é movimentar as peças até reconstruir a imagem original.

O código do jogo do quebra-cabeça pode ser encontrado neste [link](#). Já as imagens usadas podem ser acessadas [aqui](#). Esse exemplo foi adaptado do código desenvolvido pelo aluno Junior do Lab Penguin [4].



```
import cv2
import time
import random
import numpy as np
import threading
import queue
import os
from sklearn.cluster import KMeans

# = Modalidade dos Jogos ==
game = 'Demo'
# game = 'Embaralha'

base_path = "./quebracabeca/" # Pasta das imagens
imagens = [cv2.imread(os.path.join(base_path, f"mapa_{i}.png")) for i in range(1, 16)]

# =====
# CONFIG WEBCAM
# =====
cap = cv2.VideoCapture(0)
frame_queue = queue.Queue(maxsize=2)
```

```

# === Thread: pegar fotos da camera ===
def tira_foto():
    while True:
        ret, frame = cap.read()
        if ret:
            if frame_queue.full():
                frame_queue.get_nowait()
            frame_queue.put_nowait(frame)
            time.sleep(0.01)

# === Thread: Processa e exibe os frames com overlay ===
def processar_frames(id_map):
    cv2.namedWindow("Quebra-Cabeça", cv2.WINDOW_NORMAL)
    resize_scale = 0.4

    while True:
        try:
            frame_full = frame_queue.get(timeout=1)
        except queue.Empty:
            continue

        frame_small = cv2.resize(frame_full, None, fx=resize_scale, fy=resize_scale)
        gray = cv2.cvtColor(frame_small, cv2.COLOR_BGR2GRAY)
        markerCorners, markerIds, _ = detector.detectMarkers(gray)

        if markerIds is not None:
            for corners, marker_id in zip(markerCorners, markerIds.flatten()):
                numero_da_pecas = id_map.get(int(marker_id))

                if numero_da_pecas is not None:
                    overlay = imagens[numero_da_pecas - 1]
                    if overlay is None: continue
                    h, w = overlay.shape[:2]
                    dst_pts = (corners[0] / resize_scale).astype(np.float32)
                    src_pts = np.array([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]],
                                       np.float32)
                    matrix = cv2.getPerspectiveTransform(src_pts, dst_pts)
                    warped = cv2.warpPerspective(overlay, matrix, (frame_full.shape[1],
                                                                    frame_full.shape[0]))
                    mask = np.zeros((frame_full.shape[0], frame_full.shape[1]),
                                    dtype=np.uint8)
                    cv2.fillConvexPoly(mask, dst_pts.astype(np.int32), 255)
                    mask_3ch = cv2.merge([mask]*3)
                    frame_full = cv2.bitwise_and(frame_full, cv2.bitwise_not(mask_3ch))
                    frame_full = cv2.add(frame_full, cv2.bitwise_and(warped, mask_3ch))

        frame_resized = cv2.resize(frame_full, None, fx=3, fy=3)
        cv2.imshow("Quebra-Cabeça", cv2.flip(frame_resized, 1))

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cv2.destroyAllWindows()

```



```

# === THREAD: VERIFICA GRADE DE ARUCOS =====
def verificar_grade():
    last_match_state = None

    while True:
        time.sleep(0.5)
        try:
            frame = frame_queue.get(timeout=1)
        except queue.Empty:
            continue

        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        all_corners, ids, _ = detector.detectMarkers(gray)

        if ids is None or len(ids) < 15:
            continue

        # logica de homografia e grade
        side_len = 150.0
        output_size = (int(side_len * 10), int(side_len * 10))
        pts_src = all_corners[0][0]
        offset = np.array([output_size[0] / 2 - side_len / 2, output_size[1] / 2 - side_len / 2], dtype="float32")
        pts_dst = np.array([
            [offset[0], offset[1]], [offset[0] + side_len - 1, offset[1]],
            [offset[0] + side_len - 1, offset[1] + side_len - 1], [offset[0], offset[1] + side_len - 1]
        ], dtype="float32")
        matrix = cv2.getPerspectiveTransform(pts_src, pts_dst)
        warped_image = cv2.warpPerspective(frame, matrix, output_size)
        gray_warped = cv2.cvtColor(warped_image, cv2.COLOR_BGR2GRAY)
        warped_corners, warped_ids, _ = detector.detectMarkers(gray_warped)

        if warped_ids is None or len(warped_ids) < 15:
            continue

        grid_detectada = [[None for _ in range(4)] for _ in range(4)]
        warped_centers = np.array([c[0].mean(axis=0) for c in warped_corners])
        marker_ids_list = warped_ids.flatten()
        kmeans_rows = KMeans(n_clusters=4, random_state=0, n_init=10).fit(
            warped_centers[:, 1].reshape(-1, 1))
        row_labels = kmeans_rows.labels_
        row_centers_sorted = np.argsort(kmeans_rows.cluster_centers_[:, 0])
        row_map = {label: i for i, label in enumerate(row_centers_sorted)}
        kmeans_cols = KMeans(n_clusters=4, random_state=0, n_init=10).fit(
            warped_centers[:, 0].reshape(-1, 1))
        col_labels = kmeans_cols.labels_
        col_centers_sorted = np.argsort(kmeans_cols.cluster_centers_[:, 0])
        col_map = {label: i for i, label in enumerate(col_centers_sorted)}

        for i, marker_id in enumerate(marker_ids_list):
            row = row_map[row_labels[i]]
            col = col_map[col_labels[i]]
            if 0 <= row < 4 and 0 <= col < 4:

```

```

        grid_detectada[row][col] = marker_id

    match = (expected == grid_detectada)

    if match:
        print("Sequencia correta!")
    else:
        print("Sequencia incorreta!")

if game == 'Demo':
    expected = [[1, 2, 3, 4], [5, 6, 7, 8], [9,10,11,12], [13,14,15,None]]
else:
    numeros = list(range(1, 16))
    random.shuffle(numeros)
    numeros.append(None)
    expected = [numeros[i*4:(i+1)*4] for i in range(4)]
    print("Grade esperada:")
    for row in expected:
        print(row)

id_para_pecas_map = {}
peca_numero = 1
for r in range(4):
    for c in range(4):
        if peca_numero > 15: break
        id_aruco = expected[r][c]
        if id_aruco is not None:
            id_para_pecas_map[id_aruco] = peca_numero
            peca_numero += 1

parameters = cv2.aruco.DetectorParameters()
parameters.cornerRefinementMethod = cv2.aruco.CORNER_REFINE_SUBPIX
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
detector = cv2.aruco.ArucoDetector(dictionary, parameters)

frame_queue = queue.Queue(maxsize=2)

t1 = threading.Thread(target=tira_foto, daemon=True)
t2 = threading.Thread(target=processar_frames, args=(id_para_pecas_map,))
t3 = threading.Thread(target=verificar_grade, daemon=True)

for t in [t1, t2, t3]: t.start()
t2.join()

```

Nas linhas de código destacadas, são apresentadas duas formas de execução do jogo: o modo de demonstração e o modo embaralhado. No modo de demonstração, as peças do quebra-cabeça são exibidas já organizadas na ordem correta, o que facilita a visualização do funcionamento do sistema. Já no modo embaralhado, as peças são distribuídas aleatoriamente, permitindo iniciar ou reiniciar o jogo sem a necessidade de reorganizar fisicamente os marcadores.

A escolha entre os modos é feita diretamente no código, por meio de comentários: deve-se comentar a opção que não será utilizada e manter ativa aquela desejada. No exemplo apresentado, o modo de demonstração está habilitado.

```
# = Modalidade dos Jogos ==
game = 'Demo'
# game = 'Embaralha'

base_path = "../quebracabeca/" # Pasta das imagens
imagens = [cv2.imread(os.path.join(base_path, f"mapa_{i}.png")) for i in range(1, 16)]
```

Nessa parte do código, você define onde estão as imagens usadas nos ArUcos. No exemplo, elas ficam em uma pasta chamada *quebracabeca*, que está na mesma pasta do programa em Python.

Também é importante que os arquivos sigam um padrão de nome: *mapa_número* (por exemplo, *mapa_1*, *mapa_2*, etc.). Isso é necessário para que o programa consiga encontrar e carregar as imagens corretamente.

Quando cansar de jogar, para encerrar o programa, pressione a tecla “q”.

Caso queira utilizar uma imagem diferente da disponível na [pasta exemplo](#), basta usar outra imagem quadrada e reparti-la em 16 pedaços, nomeando cada parte como *mapa_número*. Para tornar esse processo mais simples, especialmente em atividades em sala de aula, disponibilizamos um programa que realiza automaticamente o corte da imagem e salva as 16 partes na mesma pasta. Assim, o professor não precisa fazer essa etapa manualmente. O código pode ser acessado [aqui](#).

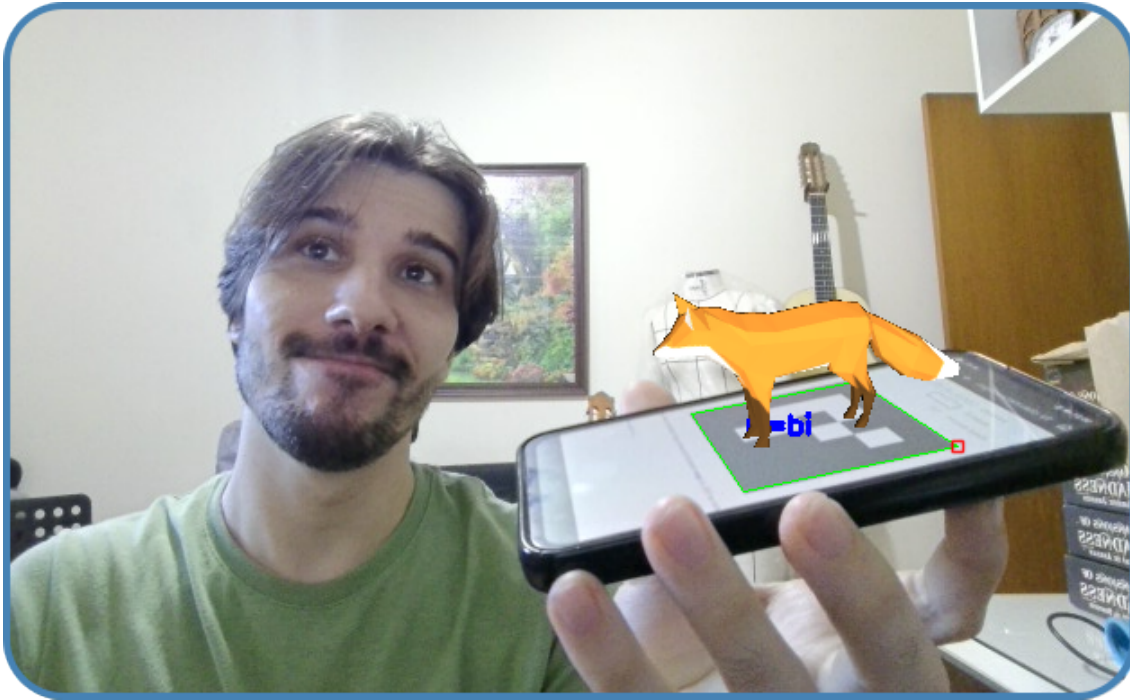


Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF01MA09 (Álgebra)** — Organização e ordenação de objetos por meio de atributos, como forma e medida. A prática exige que o estudante organize os marcadores seguindo a sequência numérica dos IDs (de 1 a 15) e a continuidade visual da figura, o que estimula o raciocínio lógico e a criação de estratégias passo a passo para resolver o problema proposto.
- **EF04MA16 (Geometria)** — Descrição de deslocamentos e localização de objetos no espaço por meio de malhas quadriculadas e representações como desenhos. No jogo, inspirado no clássico *15 Puzzle*, o aluno deve planejar o movimento dos marcadores ArUco dentro de uma grade 4×4 para reconstruir a imagem fragmentada, compreendendo as relações de posição entre as peças e o espaço vazio.



3.9 Realidade Aumentada com ArUcos



Neste exemplo, vamos criar uma aplicação de Realidade Aumentada (RA) utilizando marcadores ArUco. A ideia é fazer com que objetos e cenários 3D apareçam sobre a imagem capturada pela câmera, criando um ambiente virtual interativo.

Cada objeto mostrado na tela ficará associado a um ArUco específico. Assim, quando um marcador for identificado pela câmera, o sistema exibirá automaticamente o modelo 3D correspondente sobre ele.

Para montar os cenários, podem ser utilizados modelos 3D .obj gratuitos encontrados em sites como o [Sketchfab](https://sketchfab.com/). Muitos modelos são disponibilizados no formato .glb. Caso o formato .obj não esteja disponível, é possível converter o arquivo utilizando ferramentas online, como o site convert3d.org. Com isso, é possível criar diferentes ambientes de Realidade Aumentada utilizando múltiplos ArUcos, objetos 3D e até sons ambientes para tornar a experiência mais imersiva.



O código do exemplo de RA pode ser encontrado neste [link](#). Já os modelos 3D usados podem ser acessados [aqui](#).

```
import cv2
import numpy as np
import trimesh
import pyrender
from PIL import Image
```

```
# =====
# CAMERA
```

```

# =====
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
height, width = frame.shape[:2]

# =====
# ARUCO
# =====
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
detector = cv2.aruco.ArucoDetector(aruco_dict)

# =====
# MATRIZ DA CAMERA
# =====
fx = width
fy = width

cx = width / 2
cy = height / 2

camera_matrix = np.array([
    [fx, 0, cx],
    [0, fy, cy],
    [0, 0, 1]
], dtype=np.float32)

dist_coeffs = np.zeros((4, 1))

# =====
# TAMANHO REAL DO ARUCO
# =====
marker_size = 0.05

obj_points = np.array([
    [-marker_size/2, marker_size/2, 0],
    [ marker_size/2, marker_size/2, 0],
    [ marker_size/2, -marker_size/2, 0],
    [-marker_size/2, -marker_size/2, 0]
], dtype=np.float32)

# =====
# CENA
# =====
scene = pyrender.Scene(bg_color=[0, 0, 0, 0])

# =====
# CAMERA 3D
# =====
camera = pyrender.IntrinsicsCamera(fx=fx, fy=fy, cx=cx, cy=cy)
cam_node = scene.add(camera)

# =====
# LUZ
# =====

```

```

light = pyrender.DirectionalLight(color=np.ones(3),intensity=20.0)
scene.add(light)

# =====
# RENDERER
# =====
renderer = pyrender.OffscreenRenderer(width,height)

# =====
# FUNCAO PARA PREPARAR OBJETOS
# =====
def carregar_obj(caminho,escala=0.01,rot_x=0,rot_y=0,rot_z=0,altura=0.02,
max_textura=2048):
    mesh = trimesh.load(caminho)
    if isinstance(mesh, trimesh.Scene):
        mesh = trimesh.util.concatenate(
            tuple(mesh.geometry.values())
        )

    try:
        if hasattr(mesh.visual, 'material'):
            material = mesh.visual.material
            if hasattr(material, 'image'):
                img = material.image
                if img is not None:
                    img_np = np.array(img)
                    h, w = img_np.shape[:2]
                    print(f"Textura original: {w}x{h}")
                    # se textura muito grande
                    if w > max_textura or h > max_textura:
                        print("Redimensionando textura...")
                        pil_img = Image.fromarray(img_np)
                        pil_img.thumbnail((max_textura, max_textura))

                        material.image = np.array(pil_img)
                        nh, nw = material.image.shape[:2]
                        print(f"Nova textura: {nw}x{nh}")
    except Exception as e:
        print("Erro ao processar textura:")
        print(e)

    # centraliza
    mesh.apply_translation(-mesh.centroid)

    # =====
    # ROTACOES
    # =====
    rx = trimesh.transformations.rotation_matrix(np.radians(rot_x),[1, 0, 0])
    ry = trimesh.transformations.rotation_matrix(np.radians(rot_y),[0, 1, 0])
    rz = trimesh.transformations.rotation_matrix(np.radians(rot_z),[0, 0, 1])

    mesh.apply_transform(rx)
    mesh.apply_transform(ry)
    mesh.apply_transform(rz)

```

```
# =====
# MOVE PRA CIMA
# =====
mesh.apply_translation([0, 0, altura])

# =====
# ESCALA
# =====
mesh.apply_scale(escala)

return pyrender.Mesh.from_trimesh(mesh)

# =====
# CARREGA OBJETOS
# =====
meshes = {}

# Primeiro ArUco ID 0
meshes[0] = carregar_obj(
    "./cenario/fox/low-poly-fox-by-pixelmannen.obj",
    escala=0.0005,
    rot_x=90,
    altura=40
)

# Segundo ArUco ID 1
meshes[1] = carregar_obj(
    "./cenario/rio/low_poly_river.obj",
    escala=0.02,
    rot_x=90,
    altura=0
)

# Terceiro ArUco ID 2
meshes[2] = carregar_obj(
    "./cenario/praias/the_beach.obj",
    escala=0.02,
    rot_x=90,
    altura=0
)

# Quarto ArUco ID 3
meshes[3] = carregar_obj(
    "./cenario/birds/birds.obj",
    escala=0.02,
    rot_x=180,
    altura=5
)

# Quinto ArUco ID 4
meshes[4] = carregar_obj(
    "./cenario/chuva/rain_1.obj",
    escala=0.0002,
```

```

    rot_x=90,
    altura=1000
)

# Sexto ArUco ID 5
meshes[5] = carregar_obj(
    "./cenario/fogueira/printable_fire_pit.obj",
    escala=0.01,
    rot_x=90,
    altura=0
)

# Setimo ArUco ID 6
meshes[6] = carregar_obj(
    "./cenario/vila/village_low_poly.obj",
    escala=0.1,
    rot_x=90,
    altura=0,
    max_textura=4096
)

# Oitavo ArUco ID 7
meshes[7] = carregar_obj(
    "./cenario/barco/rowing_boat.obj",
    escala=0.00002,
    rot_x=90,
    altura=1000
)

# =====
# CRIA NODES FIXOS
# =====
nodes = {}
for marker_id, mesh in meshes.items():
    node = scene.add(mesh, pose=np.eye(4))
    nodes[marker_id] = node

# =====
# LOOP
# =====
while True:

    ret, frame = cap.read()
    if not ret:
        break

    corners, ids, _ = detector.detectMarkers(frame)

    # =====
    # ESCONDE OBJETOS
    # =====
    for node in nodes.values():
        scene.set_pose(node,
            np.array([

```



```

        [1,0,0,1000],
        [0,1,0,1000],
        [0,0,1,1000],
        [0,0,0,1]
    ])
)

# =====
# DETECCAO
# =====
if ids is not None:
    cv2.aruco.drawDetectedMarkers(frame, corners, ids)
    for i, marker_id in enumerate(ids.flatten()):
        marker_id = int(marker_id)
        img_points = corners[i][0]
        success, rvec, tvec = cv2.solvePnP(obj_points, img_points, camera_matrix,
        dist_coeffs)
        if success:
            rot_mat, _ = cv2.Rodrigues(rvec)
            pose = np.eye(4)
            pose[:3, :3] = rot_mat
            pose[:3, 3] = tvec.flatten()

            # =====
            # CORRIGE EIXOS OPENGL
            # =====
            pose[1, :] *= -1
            pose[2, :] *= -1

            # =====
            # ATUALIZA POSE DO NODE
            # =====
            if marker_id in nodes:
                scene.set_pose(nodes[marker_id], pose)

# =====
# RENDERIZA UMA VEZ
# =====
color, depth = renderer.render(scene)

# RGB -> BGR
color = color[:, :, ::-1]

# mascara
mask = depth > 0

# mistura
frame[mask] = color[:, :, :3][mask]

cv2.imshow("Realidade Aumentada com ArUcos", cv2.flip(frame, 1))

key = cv2.waitKey(1)
if key == ord('q'):
    break

```

```
# =====
# FINALIZA
# =====
cap.release()
cv2.destroyAllWindows()
```

No trecho abaixo, definimos o tamanho real do marcador ArUco utilizado no projeto. O valor 0.05 indica que o marcador possui aproximadamente 5 centímetros de lado. Essa informação é importante, pois influencia diretamente a escala e o posicionamento dos objetos 3D na cena de Realidade Aumentada. Caso o valor configurado seja muito diferente do tamanho real do ArUco impresso, os objetos podem aparecer desproporcionais, muito grandes, muito pequenos ou posicionados de maneira incorreta sobre o marcador.

```
# =====
# TAMANHO REAL DO ARUCO
# =====
marker_size = 0.05
```

Já no recorte abaixo, carregamos o objeto 3D *village_low_poly.obj*, presente na pasta */cenario/vila/*, dentro da pasta do projeto. Esse modelo será exibido sempre que o ArUco de ID 6 for identificado pela câmera. Caso queira mudar o ID do ArUco, basta colocar outro número que não o 6 em `meshes[6]`.

Os parâmetros utilizados ajudam a deixar o objeto do jeito desejado na cena. O valor de escala controla o tamanho do modelo 3D, `rot_x` ajusta sua rotação para que ele fique na orientação correta, e `altura` move o objeto para cima ou para baixo em relação ao marcador. Também é possível rotacionar o modelo 3D nos eixos *y* e *z*, utilizando os parâmetros `rot_y` e `rot_z` no código. Nos exemplos apresentados, alguns modelos foram configurados com rotação de 90° porque foram baixados em uma orientação diferente da esperada, aparecendo “deitados” na cena de Realidade Aumentada (faça o teste: troque 90 por 0 e rode o programa).

```
# Setimo ArUco ID 6
meshes[6] = carregar_obj(
    "./cenario/vila/village_low_poly.obj",
    escala=0.1,
    rot_x=90,
    altura=0,
    max_textura=4096
)
```

Já o parâmetro `max_textura` ajuda a evitar problemas com texturas muito grandes, reduzindo automaticamente seu tamanho quando necessário. Essa limitação foi adicionada porque alguns modelos 3D baixados possuíam texturas muito pesadas, o que poderia causar erros, deixar a renderização mais lenta ou fazer o modelo aparecer totalmente preto por falha no carregamento da textura (caso aconteça, tente reduzir de 4096 para 2048 ou 1024).

A organização das pastas do projeto deve seguir o exemplo mostrado na Figura 3.1. Sempre que for adicionar um novo modelo 3D, é necessário criar uma nova entrada no código para associá-lo a um ArUco (assim como mostrado anteriormente) e também colocar sua pasta dentro da pasta *cenario*.

Dentro da pasta do objeto devem estar o arquivo do modelo (*.obj*), o arquivo de materiais (*.mtl*) e a imagem da textura utilizada pelo modelo (normalmente, um *.png*).

```

exemploRA/
|
|-- exemploRA.py
|
|-- cenario/
|   |
|   |-- fox/
|   |   |-- fox.obj
|   |   |-- fox.mtl
|   |   \-- textura.png
|   |
|   \-- vila/
|       |-- vila.obj
|       |-- vila.mtl
|       \-- textura.png

```

Figura 3.1: Exemplo de organização das pastas do projeto.

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode usar ferramentas de IA para adaptar o código e criar novas experiências em Realidade Aumentada. Os ArUcos podem, por exemplo, representar peças de roupa para montar *looks* ou simular um desfile em RA. O importante é usar o exemplo como ponto de partida para experimentar novas ideias e criar aplicações próprias.

Nas próximas seções, vamos explorar outras possibilidades da Realidade Aumentada com ArUcos. Além de exibir objetos 3D, os marcadores também podem controlar sons e criar ambientes interativos. Por exemplo, é possível montar um mixer de áudio em que cada ArUco ative um som específico, além de combinar áudio e modelos 3D para criar cenários imersivos, como florestas, praias ou vilas.

E por que parar por aí? Bom, sem tantos *spoilers*... Continue lendo a apostila. >_o

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF09MA17 (Geometria)** — Reconhecimento de vistas ortogonais de figuras espaciais e aplicação desse conhecimento para desenhar objetos em perspectiva. A prática permite que o aluno visualize objetos tridimensionais sobre a imagem da câmera e compreenda como a posição e rotação do marcador físico determinam a projeção e a escala do modelo virtual na tela.
- **EF15AR26 (Música) e EF69AR03 (Artes Visuais)** — Exploração de tecnologias, recursos digitais e audiovisuais nos processos de criação artística para a construção de ambientes imersivos e interativos.
- **EF04CI04 (Vida e evolução)** — Análise e construção de cadeias alimentares simples, reconhecendo a posição ocupada pelos seres vivos nessas cadeias e o papel do Sol como fonte primária de energia na produção de alimentos. A prática permite que o estudante utilize os modelos 3D disponíveis (como a raposa e os pássaros) para organizar e compreender visualmente as relações tróficas e o fluxo de energia dentro do ecossistema criado em Realidade Aumentada.



3.10 Mixer de áudios com ArUcos



No exemplo anterior, adicionamos diversos modelos 3D à imagem capturada pela câmera, criando um cenário em Realidade Aumentada. Porém, apesar dos elementos visuais, o ambiente ainda estava silencioso.

Nesta atividade, vamos utilizar os ArUcos de uma maneira diferente: em vez de exibir objetos 3D, cada marcador será responsável por ativar um som específico. Assim, conforme diferentes ArUcos forem detectados pela câmera, os sons serão reproduzidos e misturados automaticamente, permitindo a criação de ambientes sonoros interativos.



O código do mixer de áudio pode ser encontrado neste [link](#). Já os arquivos de áudio usados podem ser acessados [aqui](#).

```
import cv2
import pygame
import numpy as np

# =====
# INICIA AUDIO
# =====

pygame.mixer.init()
pygame.mixer.set_num_channels(16)
canais = {}

for i in range(16):
    canais[i] = pygame.mixer.Channel(i)

# =====
# CARREGA SONS
```

```
# =====
sons = {}
# 0 - fox
# 1 - floresta
# 2 - praia
# 3 - passaros
# 4 - chuva
# 5 - fogueira
# 6 - vila
# 7 - barco

sons[0] = pygame.mixer.Sound("./sons/fox.wav")
sons[1] = pygame.mixer.Sound("./sons/floresta.wav")
sons[2] = pygame.mixer.Sound("./sons/praias.wav")
sons[3] = pygame.mixer.Sound("./sons/passaros.wav")
sons[4] = pygame.mixer.Sound("./sons/chuva.wav")
sons[5] = pygame.mixer.Sound("./sons/fogueira.wav")
sons[6] = pygame.mixer.Sound("./sons/vila_medieval.wav")
sons[7] = pygame.mixer.Sound("./sons/barco.wav")

# volumes
sons[0].set_volume(0.6)
sons[1].set_volume(0.6)
sons[2].set_volume(0.5)
sons[3].set_volume(0.5)
sons[4].set_volume(0.5)
sons[5].set_volume(0.5)
sons[6].set_volume(0.5)
sons[7].set_volume(0.5)

# =====
# SONS TOCANDO
# =====
tocando = {}

# =====
# CAMERA
# =====
cap = cv2.VideoCapture(0)

# =====
# ARUCO
# =====
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
detector = cv2.aruco.ArucoDetector(aruco_dict)

# =====
# LOOP
# =====
while True:
    ret, frame = cap.read()
    if not ret:
        break
    corners, ids, _ = detector.detectMarkers(frame)
```

```

# =====
# ARUCOS VISIVEIS
# =====
ids_ativos = set()
if ids is not None:
    cv2.aruco.drawDetectedMarkers(frame, corners, ids)
    ids_ativos = set(ids.flatten())

# =====
# TOCA NOVOS SONS
# =====
for marker_id in ids_ativos:
    if marker_id in sons:
        if marker_id not in tocando:
            canal = canais[marker_id]
            canal.play(sons[marker_id], loops=-1)
            tocando[marker_id] = canal
            print(f"Tocando som do ArUco {marker_id}")

# =====
# PARA SONS REMOVIDOS
# =====
for marker_id in list(tocando.keys()):
    if marker_id not in ids_ativos:
        tocando[marker_id].stop()
        del tocando[marker_id]
        print(f"Parando som do ArUco {marker_id}")

# =====
# MOSTRA CAMERA
# =====
cv2.imshow("Audio ArUco", cv2.flip(frame, 1))

key = cv2.waitKey(1)
if key == ord('q'):
    break

# =====
# FINALIZA
# =====
pygame.mixer.quit()
cap.release()
cv2.destroyAllWindows()

```

Como visto no código, foram adicionados 8 sons diferentes, associados aos ArUcos de IDs de 0 a 7. Para adicionar um novo som ligado a qualquer outro marcador, basta incluir duas novas linhas junto das demais configurações de áudio. No exemplo abaixo, foi adicionado um som de elefante armazenado na pasta /sons/. Esse som será ativado quando o ArUco de ID 10 for detectado pela câmera, utilizando volume ajustado para 0.8.

```

sons[10] = pygame.mixer.Sound("./sons/elefante.wav")
# volumes
sons[10].set_volume(0.8)

```

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como usar os ArUcos para fazer um piano virtual, uma bateria digital ou até um sistema de sons ambientes para uma aventura de RPG. Por que não?!

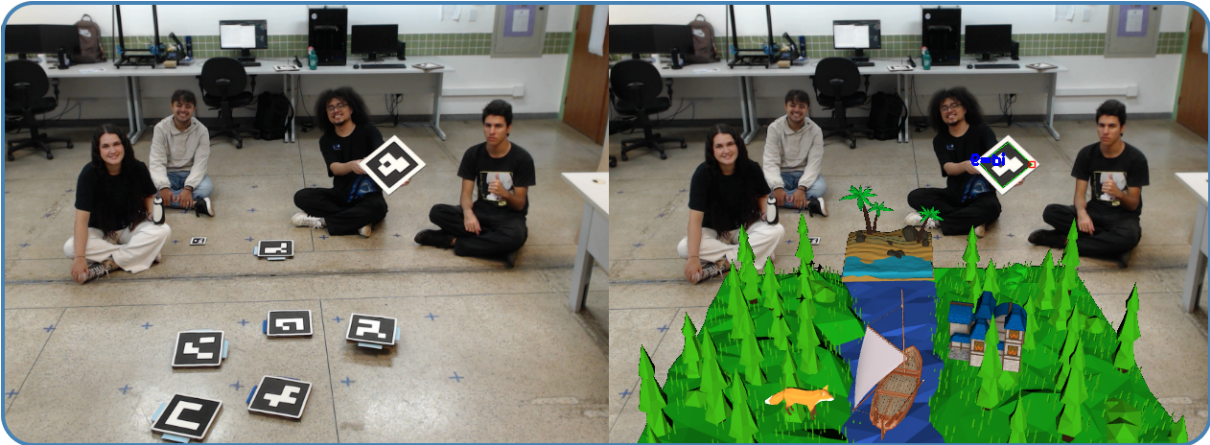
Outra ideia é utilizar sons gravados pela própria turma para criar um ambiente mais imersivo e divertido. Caso o seu dispositivo não grave diretamente em formato .wav, é possível utilizar conversores online para realizar essa conversão de forma simples.

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF15AR13 (Música)** — Identificação e apreciação de diversas formas e gêneros de expressão musical. A prática permite reconhecer as funções da música em contextos da vida cotidiana através da interação com uma interface de piano digital.
- **EF15AR26 (Artes Integradas)** — Exploração de tecnologias e recursos digitais nos processos de criação artística. Ao utilizar a visão computacional para transformar marcadores físicos em notas musicais, o aluno exerce o protagonismo na experimentação de novas ferramentas de expressão.
- **EF03CI01 (Matéria e energia)** — Produção de sons e identificação de variáveis que influem nesse fenômeno. O exercício utiliza a interface tecnológica para simular a resposta sonora de diferentes "objetos"(IDs ArUco), permitindo que o estudante explore a relação entre a manipulação física e a geração de som.
- **EF01MA09 (Álgebra)** — organização e ordenação de objetos seguindo atributos específicos. A atividade envolve ordenar os marcadores seguindo a sequência lógica da escala musical (dó, ré, mi...) associada aos IDs, estimulando o raciocínio sequencial.



3.11 Cenário em RA com ArUcos



Finalmente, vamos juntar os dois exemplos anteriores! Agora, além de adicionar modelos 3D à imagem capturada pela câmera, também vamos incluir sons para criar um cenário muito mais imersivo em Realidade Aumentada.

Neste exemplo, cada ArUco poderá ativar um objeto 3D e também um som correspondente, tornando o ambiente mais interativo e divertido. Foram utilizados os marcadores de IDs de 0 a 7, assim como nos exemplos anteriores.



O código do cenário em RA pode ser encontrado neste [link](#). Já os modelos 3D usados podem ser acessados [aqui](#) e os arquivos de áudio [aqui](#).

```
import cv2
import pygame
import numpy as np
import trimesh
import pyrender
from PIL import Image

# =====
# INICIA AUDIO
# =====
pygame.mixer.init()
pygame.mixer.set_num_channels(16)
canais = {}

for i in range(16):
    canais[i] = pygame.mixer.Channel(i)

# =====
# CAMERA
# =====
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
height, width = frame.shape[:2]
```



```

# =====
# ARUCO
# =====
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
detector = cv2.aruco.ArucoDetector(aruco_dict)

# =====
# MATRIZ DA CAMERA
# =====
fx = width
fy = width

cx = width / 2
cy = height / 2

camera_matrix = np.array([
    [fx, 0, cx],
    [0, fy, cy],
    [0, 0, 1]
], dtype=np.float32)

dist_coeffs = np.zeros((4, 1))

# =====
# TAMANHO REAL DO ARUCO
# =====
marker_size = 0.05

obj_points = np.array([
    [-marker_size/2, marker_size/2, 0],
    [ marker_size/2, marker_size/2, 0],
    [ marker_size/2, -marker_size/2, 0],
    [-marker_size/2, -marker_size/2, 0]
], dtype=np.float32)

# =====
# CENA
# =====
scene = pyrender.Scene(bg_color=[0, 0, 0, 0])

# =====
# CAMERA 3D
# =====
camera = pyrender.IntrinsicsCamera(fx=fx,fy=fy,cx=cx,cy=cy)
cam_node = scene.add(camera)

# =====
# LUZ
# =====
light = pyrender.DirectionalLight(color=np.ones(3),intensity=20.0)
scene.add(light)

# =====

```

```

# RENDERER
# =====
renderer = pyrender.OffscreenRenderer(width,height)

# =====
# FUNCAO PARA PREPARAR OBJETOS
# =====
def carregar_obj(caminho,escala=0.01,rot_x=0,rot_y=0,rot_z=0,altura=0.02,
max_textura=2048):
    mesh = trimesh.load(caminho)
    if isinstance(mesh, trimesh.Scene):
        mesh = trimesh.util.concatenate(
            tuple(mesh.geometry.values())
        )

    try:
        if hasattr(mesh.visual, 'material'):
            material = mesh.visual.material
            if hasattr(material, 'image'):
                img = material.image
                if img is not None:
                    img_np = np.array(img)
                    h, w = img_np.shape[:2]
                    print(f"Textura original: {w}x{h}")
                    # se textura muito grande
                    if w > max_textura or h > max_textura:
                        print("Redimensionando textura...")
                        pil_img = Image.fromarray(img_np)
                        pil_img.thumbnail((max_textura, max_textura))

                        material.image = np.array(pil_img)
                        nh, nw = material.image.shape[:2]
                        print(f"Nova textura: {nw}x{nh}")
    except Exception as e:
        print("Erro ao processar textura:")
        print(e)

    # centraliza
    mesh.apply_translation(-mesh.centroid)

    # =====
    # ROTACOES
    # =====
    rx = trimesh.transformations.rotation_matrix(np.radians(rot_x),[1, 0, 0])
    ry = trimesh.transformations.rotation_matrix(np.radians(rot_y),[0, 1, 0])
    rz = trimesh.transformations.rotation_matrix(np.radians(rot_z),[0, 0, 1])

    mesh.apply_transform(rx)
    mesh.apply_transform(ry)
    mesh.apply_transform(rz)

    # =====
    # MOVE PRA CIMA
    # =====

```

```
mesh.apply_translation([0, 0, altura])

# =====
# ESCALA
# =====
mesh.apply_scale(escala)

return pyrender.Mesh.from_trimesh(mesh)

# =====
# CARREGA OBJETOS
# =====
meshes = {}

# Primeiro ArUco ID 0
meshes[0] = carregar_obj(
    "./cenario/fox/low-poly-fox-by-pixelmannen.obj",
    escala=0.0005,
    rot_x=90,
    altura=40
)

# Segundo ArUco ID 1
meshes[1] = carregar_obj(
    "./cenario/rio/low_poly_river.obj",
    escala=0.02,
    rot_x=90,
    altura=-3
)

# Terceiro ArUco ID 2
meshes[2] = carregar_obj(
    "./cenario/praija/the_beach.obj",
    escala=0.02,
    rot_x=90,
    altura=0
)

# Quarto ArUco ID 3
meshes[3] = carregar_obj(
    "./cenario/birds/birds.obj",
    escala=0.02,
    rot_x=180,
    altura=5
)

# Quinto ArUco ID 4
meshes[4] = carregar_obj(
    "./cenario/chuva/rain_1.obj",
    escala=0.0002,
    rot_x=90,
    altura=1000
)
```

```

# Sexto ArUco ID 5
meshes[5] = carregar_obj(
    "./cenario/fogueira/printable_fire_pit.obj",
    escala=0.01,
    rot_x=90,
    altura=0
)

# Setimo ArUco ID 6
meshes[6] = carregar_obj(
    "./cenario/vila/village_low_poly.obj",
    escala=0.1,
    rot_x=90,
    altura=0,
    max_textura=4096
)

# Oitavo ArUco ID 7
meshes[7] = carregar_obj(
    "./cenario/barco/rowing_boat.obj",
    escala=0.00002,
    rot_x=90,
    altura=1000
)

# =====
# CARREGA SONS
# =====
sons = {}
# 0 - fox
# 1 - floresta
# 2 - praia
# 3 - passaros
# 4 - chuva
# 5 - fogueira
# 6 - vila
# 7 - barco
sons[0] = pygame.mixer.Sound("./sons/fox.wav")
sons[1] = pygame.mixer.Sound("./sons/floresta.wav")
sons[2] = pygame.mixer.Sound("./sons/praias.wav")
sons[3] = pygame.mixer.Sound("./sons/passaros.wav")
sons[4] = pygame.mixer.Sound("./sons/chuva.wav")
sons[5] = pygame.mixer.Sound("./sons/fogueira.wav")
sons[6] = pygame.mixer.Sound("./sons/vila_medieval.wav")
sons[7] = pygame.mixer.Sound("./sons/barco.wav")

# volumes
sons[0].set_volume(0.6)
sons[1].set_volume(0.6)
sons[2].set_volume(0.5)
sons[3].set_volume(0.5)
sons[4].set_volume(0.5)
sons[5].set_volume(0.5)
sons[6].set_volume(0.5)

```

```

sons[7].set_volume(0.5)

# =====
# SONS TOCANDO
# =====
tocando = {}

# =====
# CRIA NODES FIXOS
# =====
nodes = {}
for marker_id, mesh in meshes.items():
    node = scene.add(mesh,pose=np.eye(4))
    nodes[marker_id] = node

# =====
# LOOP
# =====
while True:

    ret, frame = cap.read()
    if not ret:
        break
    corners, ids, _ = detector.detectMarkers(frame)

    # =====
    # ESCONDE OBJETOS
    # =====
    for node in nodes.values():
        scene.set_pose(node,
            np.array([
                [1,0,0,1000],
                [0,1,0,1000],
                [0,0,1,1000],
                [0,0,0,1]
            ])
        )

    # =====
    # DETECCAO
    # =====
    ids_ativos = set()
    if ids is not None:
        cv2.aruco.drawDetectedMarkers(frame,corners,ids)
        ids_ativos = set(ids.flatten())
        for i, marker_id in enumerate(ids.flatten()):
            marker_id = int(marker_id)
            img_points = corners[i][0]
            success, rvec, tvec = cv2.solvePnP(obj_points,img_points,camera_matrix,
                dist_coeffs)
            if success:
                rot_mat, _ = cv2.Rodrigues(rvec)
                pose = np.eye(4)
                pose[:3, :3] = rot_mat

```

```

pose[:3, 3] = tvec.flatten()

# =====
# CORRIGE EIXOS OPENGL
# =====
pose[1, :] *= -1
pose[2, :] *= -1

# =====
# ATUALIZA POSE DO NODE
# =====
if marker_id in nodes:
    scene.set_pose(nodes[marker_id],pose)
    if marker_id not in tocando:
        canal = canais[marker_id]
        canal.play(sons[marker_id],loops=-1)
        tocando[marker_id] = canal
        print(f"Tocando som do ArUco {marker_id}")

# =====
# PARA SONS REMOVIDOS
# =====
for marker_id in list(tocando.keys()):
    if marker_id not in ids_ativos:
        tocando[marker_id].stop()
        del tocando[marker_id]
        print(f"Parando som do ArUco {marker_id}")

# =====
# RENDERIZA UMA VEZ
# =====
color, depth = renderer.render(scene)

# RGB -> BGR
color = color[:, :, ::-1]

# mascara
mask = depth > 0

# mistura
frame[mask] = color[:, :, :3][mask]

cv2.imshow("Cenario RA", cv2.flip(frame,1))

key = cv2.waitKey(1)
if key == ord('q'):
    break

# =====
# FINALIZA
# =====
pygame.mixer.quit()
cap.release()
cv2.destroyAllWindows()

```

Assim como comentado anteriormente, neste exemplo foram combinados os códigos das Seções 3.9 e 3.10. Portanto, para entender melhor como cada parte funciona, vale a pena revisar essas seções antes de continuar.

De qualquer forma, a seguir é apresentado um exemplo simples mostrando como adicionar um novo modelo 3D e também um som associado a um novo ID de ArUco.

```
meshes[47] = carregar_obj(
    "./cenario/elefante/elefante.obj",
    escala=0.002,
    rot_x=90,
    altura=1
)
sons[47] = pygame.mixer.Sound("./sons/elefante.wav")
# volumes
sons[47].set_volume(0.7)
```

Neste exemplinho, um elefante é adicionado ao ambiente utilizando um ArUco de ID 47. Os arquivos do modelo 3D foram colocados em uma pasta chamada elefante, dentro da pasta cenario, enquanto o som utilizado foi adicionado à pasta sons.

O volume do áudio foi ajustado para 0.7. Já os parâmetros de escala, rot_x e altura foram definidos por meio de testes, até que o modelo ficasse bem posicionado e com um tamanho adequado na cena de Realidade Aumentada.

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como fazer com que diferentes objetos 3D interajam automaticamente quando certos ArUcos forem detectados ao mesmo tempo. Por exemplo, é possível mover objetos para posições específicas do cenário, criar animações simples ou até desenvolver pequenos jogos interativos utilizando essa ideia em Realidade Aumentada.

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF15AR26 (Artes Integradas)** — Exploração de tecnologias e recursos digitais nos processos de criação artística. Ao compor um cenário que une modelos 3D e arquivos de áudio, o estudante experimenta a relação processual entre diferentes linguagens artísticas através da Realidade Aumentada.
- **EF15AR13 (Música)** — Identificação e apreciação de diversas formas e gêneros de expressão musical. Pela inclusão de sons associados aos IDs dos marcadores, a proposta pode trabalhar a função da sonorização na construção de ambientes imersivos.
- **EF04CI04 (Vida e Evolução)** — Análise e construção de cadeias alimentares simples, reconhecendo a posição ocupada pelos seres vivos e o fluxo de energia no ambiente. A atividade permite que o estudante manipule marcadores ArUco para posicionar modelos 3D (como a raposa e os pássaros) e ativar seus respectivos sons, construindo uma representação visual e sonora das relações tróficas dentro do ecossistema criado em Realidade Aumentada.
- **Matemática (Geometria)** — estudo de geometria espacial e projeções em um contexto aplicado. A atividade permite observar como movimentos físicos de translação e rotação do marcador ArUco no mundo real são traduzidos em coordenadas matemáticas que definem a escala e a posição dos objetos virtuais na tela.



3.12 Sistema solar em RA com ArUcos



Seguindo a mesma ideia dos exemplos anteriores, agora vamos utilizar marcadores ArUcos para trazer o sistema solar para a Realidade Aumentada. Neste exemplo, cada ArUco será responsável por representar um corpo celeste diferente do nosso sistema solar.

Como se trata apenas de uma demonstração, foram adicionados o Sol, os planetas do sistema solar (incluindo Plutão, afinal, sou dos anos 90!) e também a nossa Lua.



O código do sistema solar em RA pode ser encontrado neste [link](#). Já os modelos 3D dos astros usados podem ser acessados [aqui](#).

```
import cv2
import numpy as np
import trimesh
import pyrender
from PIL import Image

# =====
# CAMERA
# =====
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
ret, frame = cap.read()
height, width = frame.shape[:2]
```



```

# =====
# ARUCO
# =====
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
detector = cv2.aruco.ArucoDetector(aruco_dict)

# =====
# MATRIZ DA CAMERA
# =====
fx = width
fy = width

cx = width / 2
cy = height / 2

camera_matrix = np.array([
    [fx, 0, cx],
    [0, fy, cy],
    [0, 0, 1]
], dtype=np.float32)

dist_coeffs = np.zeros((4, 1))

# =====
# TAMANHO REAL DO ARUCO
# =====
marker_size = 0.05

obj_points = np.array([
    [-marker_size/2, marker_size/2, 0],
    [ marker_size/2, marker_size/2, 0],
    [ marker_size/2, -marker_size/2, 0],
    [-marker_size/2, -marker_size/2, 0]
], dtype=np.float32)

# =====
# CENA
# =====
scene = pyrender.Scene(bg_color=[0, 0, 0, 0])

# =====
# CAMERA 3D
# =====
camera = pyrender.IntrinsicsCamera(fx=fx,fy=fy,cx=cx,cy=cy)
cam_node = scene.add(camera)

# =====
# LUZ
# =====
light = pyrender.DirectionalLight(color=np.ones(3),intensity=20.0)
scene.add(light)

# =====

```

```

# RENDERER
# =====
renderer = pyrender.OffscreenRenderer(width,height)

# =====
# FUNCAO PARA PREPARAR OBJETOS
# =====
def carregar_obj(caminho,escala=0.01,rot_x=0,rot_y=0,rot_z=0,altura=0.02,
max_textura=4096):
    mesh = trimesh.load(caminho)
    if isinstance(mesh, trimesh.Scene):
        mesh = trimesh.util.concatenate(
            tuple(mesh.geometry.values())
        )

    try:
        if hasattr(mesh.visual, 'material'):
            material = mesh.visual.material
            if hasattr(material, 'image'):
                img = material.image
                if img is not None:
                    img_np = np.array(img)
                    h, w = img_np.shape[:2]
                    print(f"Textura original: {w}x{h}")
                    # se textura muito grande
                    if w > max_textura or h > max_textura:
                        print("Redimensionando textura...")
                        pil_img = Image.fromarray(img_np)
                        pil_img.thumbnail((max_textura, max_textura))

                        # material.image = np.array(pil_img)
                        material.image = pil_img
                        nw, nh = pil_img.size
                        print(f"Nova textura: {nw}x{nh}")
                    else:
                        img_np = np.fliplr(img_np)
                        # atualiza textura
                        material.image = img_np

    except Exception as e:
        print("Erro ao processar textura:")
        print(e)

    # centraliza
    mesh.apply_translation(-mesh.centroid)

    # =====
    # ROTACOES
    # =====
    rx = trimesh.transformations.rotation_matrix(np.radians(rot_x),[1, 0, 0])
    ry = trimesh.transformations.rotation_matrix(np.radians(rot_y),[0, 1, 0])
    rz = trimesh.transformations.rotation_matrix(np.radians(rot_z),[0, 0, 1])

    mesh.apply_transform(rx)

```

```

mesh.apply_transform(ry)
mesh.apply_transform(rz)

# =====
# MOVE PRA CIMA
# =====
mesh.apply_translation([0, 0, altura])

# =====
# ESCALA
# =====
mesh.apply_scale(escala)

return pyrender.Mesh.from_trimesh(mesh)

# =====
# CARREGA OBJETOS
# =====
meshes = {}

# Primeiro ArUco ID 0
meshes[0] = carregar_obj(
    "./sistemasolar/sol/sun.obj",
    escala=0.02,
    rot_x=90,
    altura=10
)

# Segundo ArUco ID 1
meshes[1] = carregar_obj(
    "./sistemasolar/mercurio/mercurio.obj",
    escala=0.02,
    rot_x=90,
    altura=2
)

# Terceiro ArUco ID 2
meshes[2] = carregar_obj(
    "./sistemasolar/venus/venus.obj",
    escala=0.03,
    rot_x=90,
    altura=1
)

# Quarto ArUco ID 3
meshes[3] = carregar_obj(
    "./sistemasolar/terra/earth (3).obj",
    escala=0.03,
    rot_x=90,
    altura=1
)

# Quinto ArUco ID 4
meshes[4] = carregar_obj(

```

```
    "./sistemasolar/marte/marte_v1.1.obj",
    escala=0.0004,
    rot_x=90,
    altura=100
)

# Sexto ArUco ID 5
meshes[5] = carregar_obj(
    "./sistemasolar/jupiter/realistic_jupiter.obj",
    escala=0.001,
    rot_x=90,
    altura=100
)

# Setimo ArUco ID 6
meshes[6] = carregar_obj(
    "./sistemasolar/saturno/saturno_saturn.obj",
    escala=0.1,
    rot_x=90,
    altura=1,
    max_textura=4096
)

# Oitavo ArUco ID 7
meshes[7] = carregar_obj(
    "./sistemasolar/urano/uranus.obj",
    escala=0.0003,
    rot_x=0,
    altura=400
)

# Nono ArUco ID 8
meshes[8] = carregar_obj(
    "./sistemasolar/netuno/neptune.obj",
    escala=0.04,
    rot_x=90,
    altura=2
)

# Decimo ArUco ID 9
meshes[9] = carregar_obj(
    "./sistemasolar/plutao/pluto.obj",
    escala=0.0002,
    rot_x=90,
    altura=100
)

# Decimo primeiro ArUco ID 10
meshes[10] = carregar_obj(
    "./sistemasolar/lua/lua.obj",
    escala=0.01,
    rot_x=90,
    altura=4
)
```

```

# =====
# CRIA NODES FIXOS
# =====
nodes = {}
for marker_id, mesh in meshes.items():
    node = scene.add(mesh, pose=np.eye(4))
    nodes[marker_id] = node

# =====
# LOOP
# =====
while True:

    ret, frame = cap.read()
    if not ret:
        break

    corners, ids, _ = detector.detectMarkers(frame)

    # =====
    # ESCONDE OBJETOS
    # =====
    for node in nodes.values():
        scene.set_pose(node,
            np.array([
                [1,0,0,1000],
                [0,1,0,1000],
                [0,0,1,1000],
                [0,0,0,1]
            ])
        )

    # =====
    # DETECCAO
    # =====
    if ids is not None:
        cv2.aruco.drawDetectedMarkers(frame, corners, ids)
        for i, marker_id in enumerate(ids.flatten()):
            marker_id = int(marker_id)
            img_points = corners[i][0]
            success, rvec, tvec = cv2.solvePnP(obj_points, img_points, camera_matrix,
                dist_coeffs)
            if success:
                rot_mat, _ = cv2.Rodrigues(rvec)
                pose = np.eye(4)
                pose[:3, :3] = rot_mat
                pose[:3, 3] = tvec.flatten()

                # =====
                # CORRIGE EIXOS OPENGL
                # =====
                pose[1, :] *= -1
                pose[2, :] *= -1

```

```

# =====
# ATUALIZA POSE DO NODE
# =====
if marker_id in nodes:
    scene.set_pose(nodes[marker_id],pose)

# =====
# RENDERIZA UMA VEZ
# =====
color, depth = renderer.render(scene)

# RGB -> BGR
color = color[:, :, ::-1]

# mascara
mask = depth > 0

# mistura
frame[mask] = color[:, :, :3][mask]

cv2.imshow("Sistema solar", cv2.flip(frame,1))

key = cv2.waitKey(1)
if key == ord('q'):
    break

# =====
# FINALIZA
# =====
cap.release()
cv2.destroyAllWindows()

```

Esse exemplo é muito similar ao programa apresentado na Seção 3.9, portanto, para entender melhor como cada parte funciona, vale a pena revisitar essa seção antes de continuar.

De qualquer forma, a seguir é apresentado um exemplo simples mostrando como adicionar um novo modelo 3D associado a um novo ID de ArUco.

```

meshes[51] = carregar_obj(
    "./sistemasolar/europa/europa_jupiter.obj",
    escala=0.06,
    rot_x=90,
    altura=4
)

```

Neste exemplinho, Europa, uma das luas de Júpiter, foi adicionada ao ambiente utilizando um ArUco de ID 51. Os arquivos do modelo 3D foram colocados em uma pasta chamada europa, dentro da pasta sistemasolar. Os parâmetros de escala, rot_x e altura foram definidos por meio de testes, até que o modelo ficasse bem posicionado e com um tamanho adequado na cena de Realidade Aumentada.

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como fazer com que os astros cujos ArUcos forem detectados comecem a girar em torno do Sol ou até rotacionem em torno de si mesmos.

Em aulas de Física ou Ciências, isso pode ser interessante para discutir temas como distâncias astronômicas, órbitas, tamanhos relativos dos planetas e movimentos dos corpos celestes.

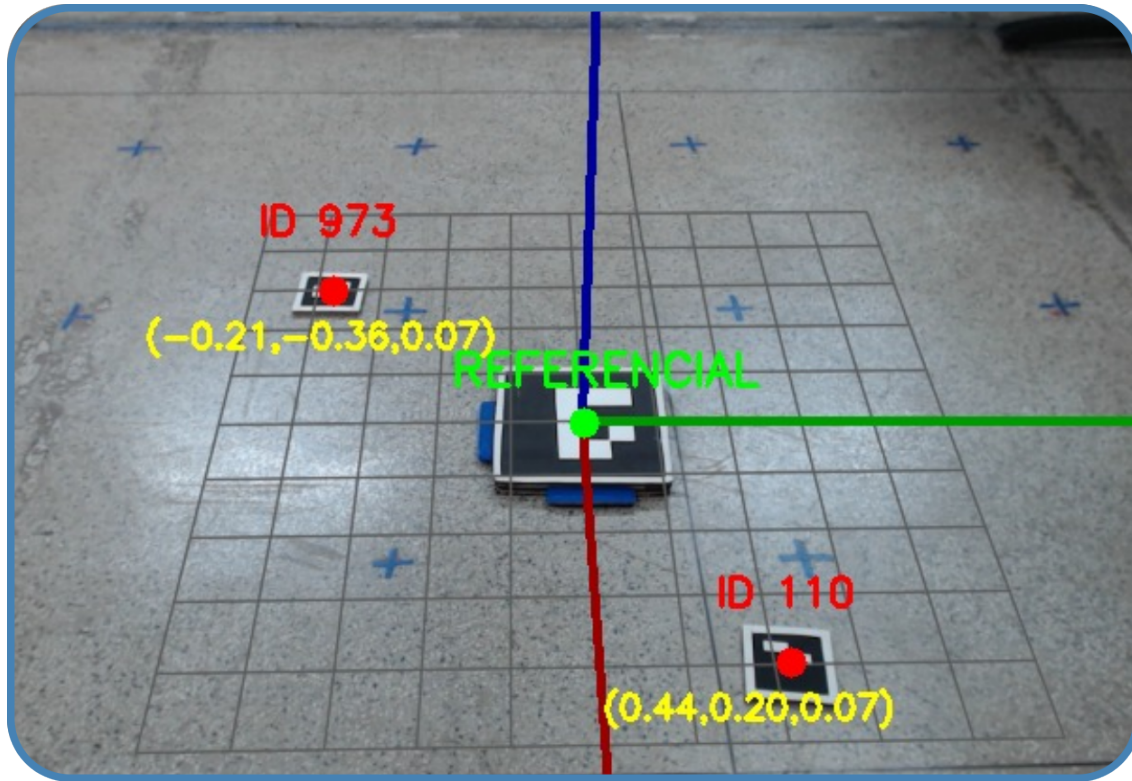
Outra ideia divertida seria adicionar audioguias, informações sobre cada planeta quando o marcador é detectado ou até criar um pequeno “planetário” interativo em Realidade Aumentada.

Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

- **EF03CI07–EF03CI08 (Terra e Universo)** — Identificação de características da Terra, como seu formato esférico, e observação dos períodos em que o Sol, a Lua e os planetas estão visíveis no céu. A prática permite que o aluno manipule representações do planeta e realize essa identificação de astros de forma digital e interativa através da Realidade Aumentada.
- **EF08CI12 (Terra e Universo)** — Justificativa de fenômenos celestes por meio da construção de modelos baseados nas posições relativas entre Sol, Terra e Lua. A utilização de marcadores físicos para representar os corpos celestes facilita a compreensão desses movimentos e posições.
- **EF01MA12 (Geometria)** — Descrição da localização de objetos no espaço segundo um dado ponto de referência. O exercício exige que o estudante descreva a localização dos astros no ambiente virtual utilizando o marcador ou sua própria posição física como referencial.
- **EF15AR26 (Artes Integradas)** — Exploração de tecnologias e recursos digitais — como a realidade aumentada e a modelagem 3D — nos processos de interação e criação artística. A atividade transforma a observação científica em uma experiência estética imersiva.



3.13 Sistema cartesiano com ArUcos



Neste exemplo, vamos utilizar marcadores ArUco para construir um sistema de coordenadas cartesianas em Realidade Aumentada. Um dos marcadores será definido como referência, servindo como origem do sistema e gerando os eixos cartesianos. Os demais ArUcos detectados serão tratados como pontos 3D, tendo suas posições calculadas em relação ao marcador de referência e exibidas na imagem em RA.



O código do sistema cartesiano em RA pode ser encontrado neste [link](#).

```
import cv2
import cv2.aruco as aruco
import numpy as np

# =====
# CONFIGURACOES
# =====

REFERENCE_ID = 1

REFERENCE_MARKER_SIZE = 0.21 # 21 cm
OBJECT_MARKER_SIZE = 0.071 # 7.1 cm

show_grid = True
xy_plane_only = True
```



```

# =====
# CAMERA
# =====
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
if not cap.isOpened():
    print("Erro ao abrir camera.")
    exit()

ret, frame = cap.read()
if not ret:
    print("Erro ao capturar frame.")
    exit()

h, w = frame.shape[:2]

# =====
# MATRIZ INTRINSECA APROXIMADA
# =====
focal_length = w
camera_matrix = np.array([
    [focal_length, 0, w / 2],
    [0, focal_length, h / 2],
    [0, 0, 1]
], dtype=np.float32)
dist_coeffs = np.zeros((5, 1), dtype=np.float32)

# =====
# ARUCO
# =====
aruco_dict = aruco.getPredefinedDictionary(aruco.DICT_4X4_1000)
parameters = aruco.DetectorParameters()
parameters.cornerRefinementMethod = (aruco.CORNER_REFINE_SUBPIX)
detector = aruco.ArucoDetector(aruco_dict, parameters)

# =====
# MODELOS 3D
# =====
ref_half = REFERENCE_MARKER_SIZE / 2
reference_obj_points = np.array([
    [-ref_half, ref_half, 0],
    [ref_half, ref_half, 0],
    [ref_half, -ref_half, 0],
    [-ref_half, -ref_half, 0]
], dtype=np.float32)

obj_half = OBJECT_MARKER_SIZE / 2
object_obj_points = np.array([
    [-obj_half, obj_half, 0],
    [obj_half, obj_half, 0],
    [obj_half, -obj_half, 0],
    [-obj_half, -obj_half, 0]
], dtype=np.float32)

# =====

```

```

# MATRIZ HOMOGENEA
# =====
def transformation_matrix(rvec, tvec):
    R, _ = cv2.Rodrigues(rvec)
    T = np.eye(4, dtype=np.float32)
    T[:3, :3] = R
    T[:3, 3] = tvec.flatten()
    return T

# =====
# GRID 3D
# =====
def draw_3d_grid(frame, rvec, tvec, grid_size=0.5, step=0.1, xy_only=False):
    R, _ = cv2.Rodrigues(rvec)
    values = np.arange(-grid_size, grid_size + step, step)
    lines = []

    # =====
    # PLANO XY
    # =====
    for y in values:
        lines.append((np.array([-grid_size, y, 0]), np.array([grid_size, y, 0])))

    for x in values:
        lines.append((np.array([x, -grid_size, 0]), np.array([x, grid_size, 0])))

    # =====
    # GRID 3D COMPLETO
    # =====
    if not xy_only:
        # XZ
        for z in values:
            lines.append((np.array([-grid_size, 0, z]), np.array([grid_size, 0, z])))
        for x in values:
            lines.append((np.array([x, 0, -grid_size]), np.array([x, 0, grid_size])))

        # YZ
        for z in values:
            lines.append((np.array([0, -grid_size, z]), np.array([0, grid_size, z])))
        for y in values:
            lines.append((np.array([0, y, -grid_size]), np.array([0, y, grid_size])))

    # =====
    # DESENHO
    # =====
    for p1, p2 in lines:
        pts_3d = np.float32([p1, p2])
        cam_pts = ((R @ pts_3d.T).T + tvec.reshape(1, 3))

        z1 = cam_pts[0][2]
        z2 = cam_pts[1][2]

        if z1 <= 0.01 or z2 <= 0.01:
            continue

```

```

pts_2d, _ = cv2.projectPoints(pts_3d,rvec,tvec,camera_matrix,dist_coeffs)
pts_2d = pts_2d.reshape(-1, 2)

pt1 = tuple(np.int32(pts_2d[0]))
pt2 = tuple(np.int32(pts_2d[1]))

if (abs(pt1[0]) > 5000 or abs(pt1[1]) > 5000 or abs(pt2[0]) > 5000 or
abs(pt2[1]) > 5000):
    continue

cv2.line(frame,pt1,pt2,(90, 90, 90),1,cv2.LINE_AA)

# =====
# EIXOS
# =====
def draw_axes(frame,rvec,tvec,size=0.10):
    axis_points_3D = np.float32([
        [0, 0, 0],
        [size, 0, 0],
        [0, -size, 0],
        [0, 0, size]
    ])

    imgpts, _ = cv2.projectPoints(axis_points_3D,rvec,tvec,camera_matrix,dist_coeffs)
    imgpts = imgpts.astype(int).reshape(-1, 2)
    origin = tuple(imgpts[0])

    # X vermelho
    cv2.line(frame,origin,tuple(imgpts[1] * 8 - 7 * imgpts[0]),(0,0,155),3)
    # Y verde
    cv2.line(frame,origin,tuple(imgpts[2] * 8 - 7 * imgpts[0]),(0,155,0),3)
    # Z azul
    cv2.line(frame,origin,tuple(imgpts[3] * 8 - 7 * imgpts[0]),(155,0,0),3)

# =====
# FILTRO
# =====
alpha = 0.7
filtered_positions = {}

# =====
# LOOP
# =====
while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
    corners, ids, _ = detector.detectMarkers(gray)

    reference_pose = None
    marker_poses = {}

```

```

# =====
# DETECCAO
# =====
if ids is not None:
    ids = ids.flatten()
    for i, marker_id in enumerate(ids):
        img_points = corners[i][0].astype(np.float32)
        if marker_id == REFERENCE_ID:
            current_obj_points = reference_obj_points
        else:
            current_obj_points = object_obj_points
        success, rvec, tvec = cv2.solvePnP(
            current_obj_points,
            img_points,
            camera_matrix,
            dist_coeffs,
            flags=cv2.SOLVEPNP_ITERATIVE
        )

        if not success:
            continue

        rvec = rvec.reshape(3, 1).astype(np.float32)
        tvec = tvec.reshape(3, 1).astype(np.float32)

        marker_poses[marker_id] = (rvec, tvec, img_points)

# =====
# REFERENCIAL
# =====
if marker_id == REFERENCE_ID:
    reference_pose = (rvec, tvec)

    if show_grid:
        draw_3d_grid(frame, rvec, tvec, grid_size=0.5, step=0.1,
                     xy_only=xy_plane_only)

    draw_axes(frame, rvec, tvec, size=0.10)

# =====
# FRAME ESPELHADO
# =====
display_frame = cv2.flip(frame.copy(), 1)

# =====
# DESENHO DOS OBJETOS
# =====
for marker_id, pose_data in marker_poses.items():
    rvec, tvec, img_points = pose_data
    center = np.mean(img_points, axis=0).astype(int)
    mirrored_center = (w - center[0], center[1])

# =====

```

```

# REFERENCIAL
# =====
if marker_id == REFERENCE_ID:
    cv2.circle(display_frame, mirrored_center, 8, (0, 155, 0), -1)
    cv2.putText(display_frame, "REFERENCIAL",
        (mirrored_center[0] - 70, mirrored_center[1] - 20),
        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 155, 0), 2)

# =====
# OBJETOS
# =====
else:
    if reference_pose is not None:
        ref_rvec, ref_tvec = reference_pose
        T_ref = transformation_matrix(ref_rvec, ref_tvec)
        T_obj = transformation_matrix(rvec, tvec)
        T_relative = (np.linalg.inv(T_ref) @ T_obj)

        position = T_relative[:3, 3]
        if marker_id not in filtered_positions:
            filtered_positions[marker_id] = position
        else:
            filtered_positions[marker_id] = (alpha*filtered_positions[marker_id]
            + (1 - alpha)*position)

        x = filtered_positions[marker_id][0]
        y = -filtered_positions[marker_id][1]
        z = filtered_positions[marker_id][2]

        cv2.circle(display_frame, mirrored_center, 8, (0, 0, 155), -1)
        text1 = f"ID {marker_id}"
        text2 = (f"({x:.2f}, "f"{y:.2f}, "f"{z:.2f})")
        cv2.putText(display_frame, text1,
            (mirrored_center[0] - 40, mirrored_center[1] - 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (155, 0, 0), 2)
        cv2.putText(display_frame, text2,
            (mirrored_center[0] - 100, mirrored_center[1] + 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 155, 0), 2)

# =====
# EXIBE
# =====
cv2.imshow("Coordenadas com ArUcos", display_frame)
key = cv2.waitKey(1)
if key == ord('q'):
    break

# =====
# GRID ON/OFF
# =====
elif key == ord('g'):
    show_grid = not show_grid
    print(f"Grid: {'ON' if show_grid else 'OFF'}")

```

```

# =====
# XY ONLY
# =====
elif key == ord('c'):
    xy_plane_only = not xy_plane_only
    print("Modo Grid:", "XY_ONLY" if xy_plane_only else "3D")

# =====
# FINALIZACAO
# =====
cap.release()
cv2.destroyAllWindows()

```

Neste exemplo, é possível habilitar ou desabilitar a visualização do *grid* com a tecla *g* e alternar entre os modos *2D* e *3D* com a tecla *c*. Além disso, o identificador do ArUco de referência e seu tamanho em metros podem ser ajustados nas variáveis correspondentes do código. Para que as coordenadas sejam calculadas corretamente, também é necessário informar o tamanho real dos demais ArUcos utilizados como pontos do sistema cartesiano por meio da variável `OBJECT_MARKER_SIZE`.

```

# =====
# CONFIGURACOES
# =====
REFERENCE_ID = 1

REFERENCE_MARKER_SIZE = 0.21    # 21 cm
OBJECT_MARKER_SIZE = 0.071     # 7.1 cm

```

Para encerrar o programa, pressione a tecla “q”.

Vamos brincar um pouco Tente ir um pouco além do exemplo apresentado. Você pode, por exemplo, colar o código em uma IA e pedir adaptações, como fazer aparecer funções baseadas nos pontos detectados ou até figuras geométricas.



Conexão com a BNCC Algumas habilidades que podem ser trabalhadas por meio da atividade:

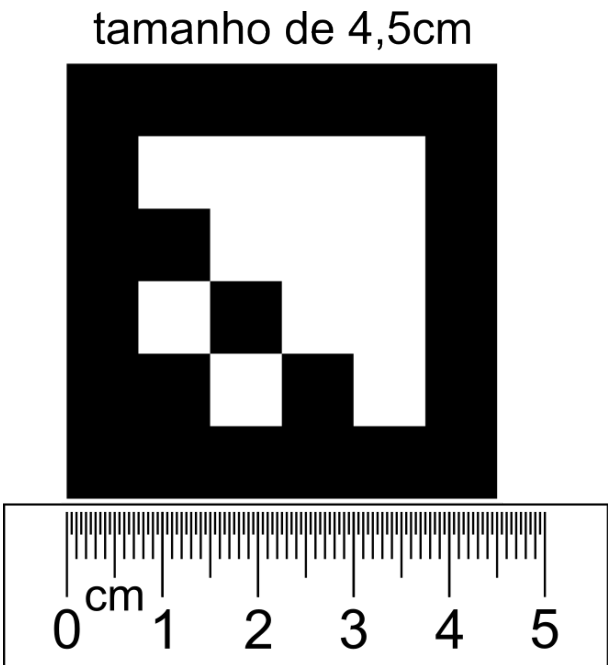
- **EF05MA15 (Geometria)** — Interpretação, descrição e representação da localização ou movimentação de objetos no plano cartesiano, utilizando coordenadas cartesianas. Ao definir um marcador ArUco como "referencial", o aluno compreende na prática o conceito de **origem (0,0)** e observa a variação das coordenadas nos eixos X, Y e Z dos demais pontos detectados no espaço.
- **EM13MAT401 e EM13MAT501** — investigação de relações numéricas e conversão de representações algébricas de funções em representações geométricas no plano cartesiano com o apoio de tecnologias digitais. A prática permite ao professor propor desafios como a criação de funções ou figuras geométricas baseadas nos pontos (IDs) detectados pela câmera em tempo real.

4. FAQ - Perguntas que podem surgir

4.1	Como sei qual o tamanho do ArUco?	93
4.2	Como posso gerar um ArUco?	93
4.3	Minha câmera não está abrindo, o que faço?	94
4.4	Testei o exemplo e deu algum erro, o que aconteceu?	94
4.5	A IA sugeriu uma mudança para o meu código. Quando coleí, algumas linhas do código no editor de Python (VS Code) ficaram sublinhadas de vermelho, o que aconteceu?	94
4.6	Como baixar os arquivos do Github?	95

4.1 Como sei qual o tamanho do ArUco?

O tamanho do ArUco é o tamanho do quadrado inteiro do marcador, não de partes internas. Ele define o tamanho real que ele terá quando for impresso. Se o seu ArUco já está impresso, pode-se usar uma régua para medir esse tamanho.



4.2 Como posso gerar um ArUco?

Podemos usar programas em Python ou até mesmo sites, como este [LINK](#).

4.3 Minha câmera não está abrindo, o que faço?

Se o terminal apresentar algum erro, uma boa prática é copiar a mensagem de erro e colar em uma IA junto com o código, pois isso ajuda a identificar a causa do problema.

Em muitos casos, a solução é simples e pode ser apenas alterar a forma como a câmera é inicializada dessa:

```
cap = cv2.VideoCapture(0)
```

para esta:

```
cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
```

Caso ainda não funcione, é possível que a câmera esteja em outro índice. Nesse caso, teste valores diferentes, como 1:

```
cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
```

4.4 Testei o exemplo e deu algum erro, o que aconteceu?

Pode ser que a versão do Python ou das bibliotecas instaladas não seja compatível com a utilizada na criação da apostila. Nesse caso, você pode usar uma IA para ajudar a atualizar o código ou para identificar as versões corretas das bibliotecas necessárias, além de orientar como instalá-las corretamente.

Outro ponto importante é conferir os nomes dos arquivos e variáveis. Pequenos detalhes podem causar erros, como usar acentos ou digitar nomes incorretamente. Também vale verificar se os arquivos usados no projeto estão na pasta correta.

4.5 A IA sugeriu uma mudança para o meu código. Quando coleí, algumas linhas do código no editor de Python (VS Code) ficaram sublinhadas de vermelho, o que aconteceu?

Lembre-se que para o Python, a indentação, ou seja, o os espaços no início das linhas são essenciais para o funcionamento correto do programa. Provavelmente, você terá que corrigir esse espaçamento. Uma dica é olhar como ficou o código apresentado pela IA e ver se após colado, o seu ficou igualmente espaçado. Uma alternativa é sempre pedir para a IA entregar o código completo com as alterações, assim, ela entrega o código corretamente indentado.

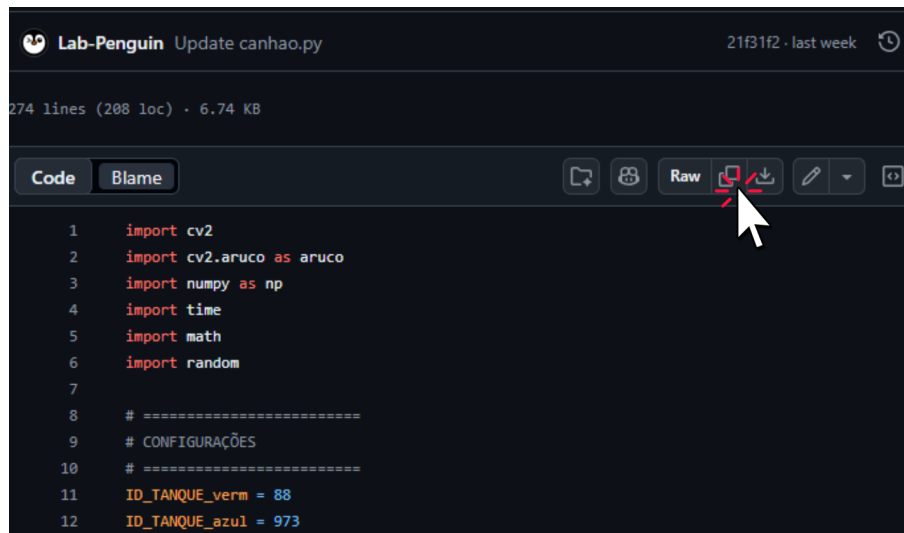
```
# Como a IA sugeriu para adicionar dentro de um for
if ID == 1:
    print("ID 1.")

# Como ficou apos colado
for ID in IDs:
    if ID == 1:
        print("ID 1.")

# Corretamente indentado
for ID in IDs:
    if ID == 1:
        print("ID 1.")
```

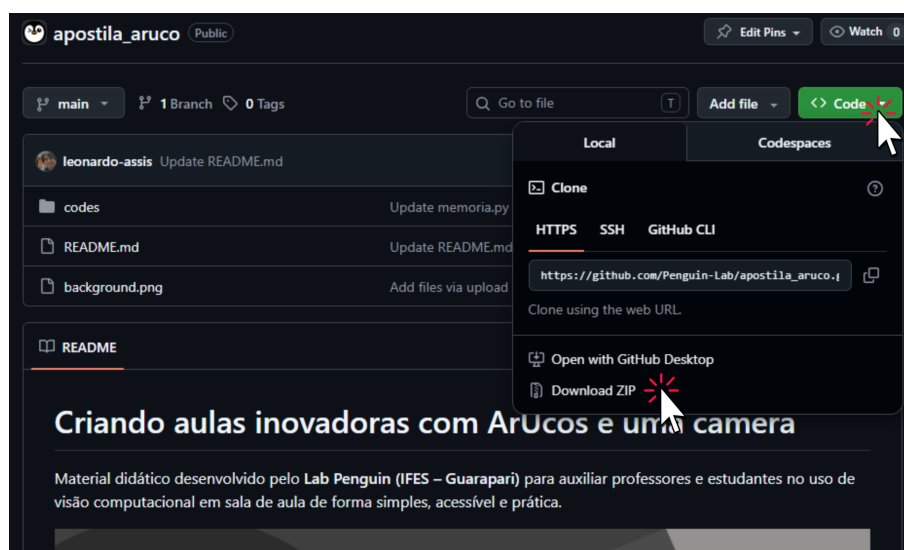

4.6 Como baixar os arquivos do Github?

Os arquivos disponibilizados no Github podem ser acessados de diferentes maneiras. Caso queira apenas visualizar ou copiar um código específico, não é necessário realizar nenhum download. Basta entrar na pasta desejada, abrir o arquivo e copiar o conteúdo diretamente da página, utilizando a seleção manual do texto ou o botão de copiar disponível no próprio Github, como mostrado abaixo.



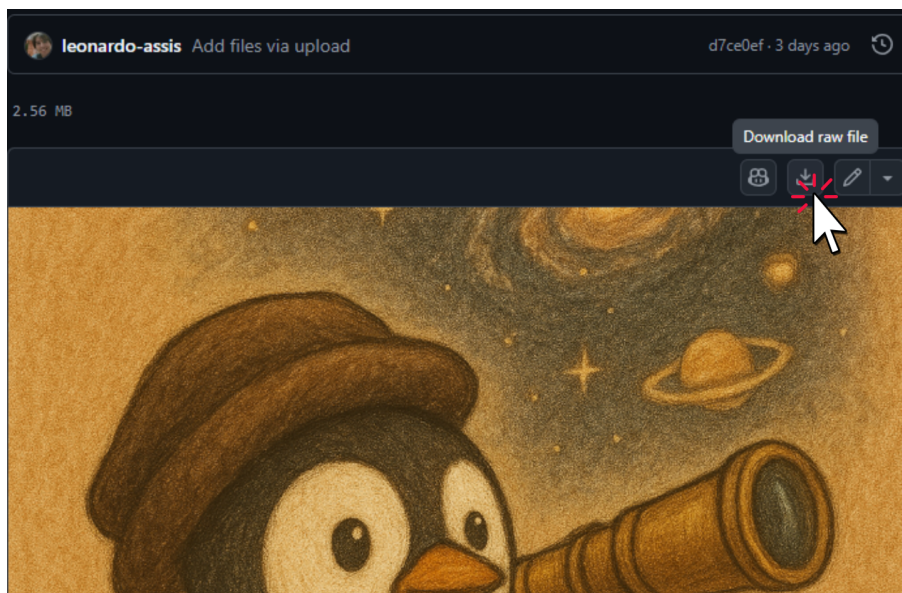
Caso deseje fazer o download dos arquivos, existem duas possibilidades: baixar todo o repositório da apostila ou baixar apenas um arquivo específico. O Github não permite baixar diretamente apenas uma pasta isolada do repositório.

A figura seguinte apresenta o procedimento para baixar todos os arquivos do projeto de uma única vez.



Já a próxima figura mostra como baixar apenas um arquivo específico. Nesse caso, é

necessário abrir o arquivo desejado no Github para que a opção de download apareça no menu superior da página.





A maior parte dos códigos desta apostila foi gerada por inteligência artificial. O objetivo é mostrar que é possível transformar ideias em implementações completas de forma acessível a qualquer pessoa.

Esperamos que tenha curtido!

>_o Equipe do Lab Penguin

- [1] Brasil. Ministério da Educação. *Base Nacional Comum Curricular*. Acesso em: 15 maio 2026. Brasília, DF, 2018. URL: https://www.gov.br/mec/pt-br/escola-em-tempo-integral/BNCC_EI_EF_110518-versaofinal.pdf.
- [2] Curso em Vídeo. *PyCharm mudou! Saiba como treinar Python no PC, celular e tablet*. Vídeo no YouTube. 2026. URL: <https://youtu.be/XND4VeMMzqg> (acesso em 22/05/2026).
- [3] jayantjain100. *Augmented-Reality*. <https://github.com/jayantjain100/Augmented-Reality>. 2021.
- [4] Junior Guimarães de Oliveira. *ArUco Puzzle Game*. <https://github.com/JuniorOliveir4/puzzle-aruco/>. 2025.
- [5] Lab Penguin. *Lab Penguin*. <https://www.eletoifes.com.br/penguin>. 2020.
- [6] Python Tutoriais UFES. *Python Tutoriais UFES*. Canal no YouTube. 2024. URL: <https://www.youtube.com/@PythonTutoriaisUFES> (acesso em 22/05/2026).
- [7] UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO. *Pro_Gramming*. Engenharia Elétrica – UFES, 2023. URL: <https://programming.ufes.br/tutor> (acesso em 22/05/2026).
- [8] Lab Visio. *aruco_draw*. https://github.com/labvisio/kits-demonstracao-visao/blob/main/aruco_draw.py. 2025.

Índice Remissivo



ArUco, [93](#)

Baixar do Github, [95](#)

Base Nacional Comum Curricular, [10](#)

BNCC, [10](#)

Câmera, [94](#)

Desenhar, [26](#)

Erro, [94](#)

Gerar ArUcos, [93](#)

Github, [95](#)

Indentação, [94](#)

Instalar bibliotecas, [14](#)

Instalar bibliotecas com IA, [15](#)

Instalação, [11](#)

Jogo, [29](#), [44](#), [48](#), [53](#)

Obras de arte, [36](#), [40](#)

Perspectiva, [40](#)

pip3, [14](#)

pip3 install, [14](#)

Plano cartesiano, [86](#)

Problema com câmera, [94](#)

Problema sugestão da IA, [94](#)

Python, [12](#)

RA, [58](#), [70](#), [78](#), [86](#)

Realidade Aumentada, [58](#), [70](#), [78](#), [86](#)

Sistema Solar, [78](#)

Sons, [66](#)

Tamanho do ArUco, [93](#)

VS Code, [16](#), [18](#)

